

STRINGS MASTER NOTES

FAANG Placement • LeetCode • Codeforces • OA Rounds • C++ Edition

Structured for Pattern Recognition • Interview Communication • Contest Thinking

40 Patterns • Brute -> Better -> Optimal • C++17

QUICK CHEAT SHEET

The fastest possible refresher before walking into an interview.

If you see this...	...reach for this
Need $O(1)$ lookup on fixed alphabet	int freq[26] array (Pattern 02)
Longest/shortest contiguous substring w/ condition	Sliding Window (Pattern 05)
Symmetric string check	Two Pointer / Expand-Around-Center (03, 04)
Same characters, different arrangement	Frequency signature / Anagram (02, 08)
Repeated prefix/word lookups at scale	Trie (Pattern 18)
Exact pattern search, guaranteed $O(n+m)$	KMP (Pattern 15)
Pattern search, multiple patterns, average case ok	Rabin-Karp rolling hash (13, 14)
True $O(n)$ longest palindromic substring	Manacher's Algorithm (Pattern 17)
Compare two strings, transform one to another	DP on Strings / Edit Distance (25, 28)
Order-preserving but skippable comparison	LCS / Subsequence DP (26, 31)
Contiguous shared run between two strings	Longest Common Substring (Pattern 27)
Segment string into dictionary words	Word Break DP (Pattern 29)
Smallest window containing all of another string	Min Window Substring + formed-counter (30)
Nested brackets / recursive-looking structure	Stack (Pattern 35)
Lexicographically smallest with greedy undo	Monotonic Stack + last-occurrence (33)
Fixed small alphabet presence/parity check	Bitmask (Pattern 38)
Feasibility of a length is monotonic	Binary Search on Answer (Pattern 37)
String problem that's secretly a graph	Decompose first: BFS/Topo-sort + string (40)

GOLDEN DECISION RULES

How to choose an algorithm family in the first 30 seconds.

STEP 1 — Classify the ask

- Existence/boolean question? -> two pointer or hashing is usually enough.
- Optimal VALUE (longest/shortest/min-ops)? -> sliding window or DP.
- ALL valid answers/combinations? -> backtracking, prune with precomputed validity.
- Repeated queries against the same fixed string/dictionary? -> preprocess: Trie, prefix arrays, suffix structures.
- Feels like it should be a graph? -> stop coding, draw the graph first (Pattern 40).

STEP 2 — Substring vs Subsequence (say this out loud, always)

- Substring = contiguous. Subsequence = order-preserved but skippable.
- This single distinction determines whether DP transitions reset-to-zero (substring) or take a max/skip (subsequence).

STEP 3 — Escalation ladder (don't over-engineer)

- 1. Brute force -- state its complexity and WHY it's too slow.
- 2. Better -- usually hashing/sorting removes one factor.
- 3. Optimal -- two pointer, sliding window, DP, or a named algorithm (KMP/Manacher/Z).
- Only escalate to suffix arrays / Manacher / Z-algorithm when explicitly pushed for true $O(n)$ -- leading with these unprompted can look like over-engineering.

TIME COMPLEXITY QUICK TABLE

Pattern Family	Typical Time Complexity
Traversal / Frequency Count	$O(n)$
Two Pointer / Sliding Window	$O(n)$
Palindrome (expand-around-center)	$O(n^2)$
Manacher's Algorithm	$O(n)$
KMP / Z-Algorithm	$O(n+m)$
Rabin-Karp	$O(n+m)$ average
Suffix Array construction	$O(n \log n)$
Trie insert/search	$O(k)$, k = word length
LCS / Edit Distance / Word Break	$O(n*m)$ or $O(n^2)$
Backtracking (partitions/combinations)	Exponential, pruned in practice
Binary Search on Answer + rolling hash	$O(n \log n)$

SPACE COMPLEXITY QUICK TABLE

Pattern Family	Typical Space Complexity
Fixed alphabet frequency array	$O(1)$ -- e.g. <code>int[26]</code>
HashMap-based signature/grouping	$O(n)$
2D DP table (LCS, Edit Distance)	$O(n*m)$, optimizable to $O(\min(n,m))$
Trie over a dictionary	$O(\text{total characters} * \text{alphabet size})$
Suffix Array + LCP array	$O(n)$
Sliding window state	$O(k)$, k = distinct characters tracked
Backtracking recursion depth	$O(n)$

FAANG FREQUENCY TABLE

How often each pattern family shows up across FAANG-style interview loops.

Pattern Family	FAANG Frequency
Sliding Window	Extremely High
Frequency Count / Anagram	Very High
Two Pointer	Very High
DP on Strings (LCS/Edit Distance/Word Break)	Very High
Trie	Very High
Hashing	Very High
Palindrome	Very High
Minimum Window Substring	Very High
Backtracking on Strings	High
Pattern Matching (KMP/Rabin-Karp)	High-Medium
Bit Manipulation + String	Medium
Manacher's / Z-Algorithm / Suffix Array	Medium (CP-heavy / senior rounds)

COMPANY WISE FREQUENCY

Which pattern families each company leans on most in string-focused rounds.

Company	Typical Emphasis
Google	KMP/Z/Manacher, Trie, DP on Strings -- values algorithmic depth and complexity proofs.
Meta	Sliding Window, Hashing, Anagram-family -- values clean, fast, communicative coding.
Amazon	Two Pointer, Sliding Window, Backtracking -- values practical, production-safe code.
Microsoft	String Simulation, DP on Strings, Trie -- values precise edge-case handling.
Apple	Two Pointer, Palindrome, String Compression -- values clean in-place solutions.
Uber	Sliding Window, Two Pointer -- real-time/matching-style constraints.
Adobe	String Compression, Sorting Strings, Simulation -- text/document processing bias.
Atlassian	Stack-based String, Simulation -- workflow/ticket text parsing style problems.
Flipkart	Anagram, Frequency Count, Sorting Strings -- e-commerce search/catalog bias.
Goldman Sachs	Sorting Strings, Simulation, Greedy String -- precise, finance-grade correctness.
Morgan Stanley	DP on Strings, Simulation -- correctness and edge-case rigor.
LinkedIn	Character Mapping, Trie -- people/entity matching style problems.
Bloomberg	Palindrome, Prefix Sum, Rolling Hash -- data-feed/text-processing bias.
Nvidia	Bit Manipulation + String, DP on Strings -- performance-oriented reasoning.
Airbnb	Sliding Window, Backtracking -- search/availability-matching style problems.
Netflix	Hashing, DP on Strings -- content/catalog matching style problems.
OpenAI	DP on Strings, Trie, Advanced synthesis problems -- research-adjacent rigor.
Databricks	Rolling Hash, Suffix structures, Prefix Sum -- large-scale text processing bias.

INTERVIEW PATTERN RECOGNITION

The master trigger-word index -- skim this in the elevator before your interview.

- "without extra space" / "in-place" -> Two Pointer, in-place read/write (01, 04, 19)
- "longest/shortest substring with..." -> Sliding Window (05, 30, 32)
- "anagram" / "same characters rearranged" -> Frequency signature (02, 08)
- "reads the same backward" -> Palindrome family (03, 17)
- "prefix" / "starts with" / "autocomplete" -> Trie, Prefix arrays (10, 18)
- "pattern occurs in text" -> KMP / Rabin-Karp / Z (13, 14, 15, 16)
- "minimum operations to transform" -> Edit Distance DP (28)
- "can be segmented into dictionary words" -> Word Break DP (29)
- "all partitions / all combinations" -> Backtracking (24)
- "nested brackets" / "decode a run-length string" -> Stack (35)
- "smallest/largest in lexicographic order" -> Monotonic Stack + greedy (33, 34)
- "feasibility of a length is monotonic" -> Binary Search on Answer (37)
- "strings define an ordering/graph" -> Decompose to graph algorithm first (40)

PATTERN 01

Basic String Traversal

FAANG Frequency	Medium
LeetCode Importance	High
Codeforces Importance	High
Companies Asking	Google, Amazon, Microsoft, Adobe, TCS/Service-based
Difficulty	Easy
Time Complexity	$O(n)$
Space Complexity	$O(1)$
Problems Covered	Reverse String, Count Vowels, Case Conversion, Char Frequency Warmup

INTUITION

- A C++ `std::string` is a contiguous, mutable char array — every advanced string pattern is built on the ability to index, slice and mutate it in $O(1)$ per access.
- Interviewers use traversal questions to see if you default to clean index-based loops or reach for unnecessary library calls that hide complexity.
- ASCII arithmetic (`c - 'a'`, `tolower/toupper` via bit tricks) is expected knowledge — not something you look up mid-interview.

PATTERN RECOGNITION — Trigger Words

- process each character
- in-place modification
- reverse / rotate the string
- count something character by character

If interviewer says “reverse this string without extra space” — think two-pointer swap, not a new string.

DRY RUN

```
s = "hello"
i=0 -> 'h'  i=1 -> 'e'  i=2 -> 'l'  i=3 -> 'l'  i=4 -> 'o'
Each index visited once,  $O(1)$  work per index ->  $O(n)$  total.
```

BRUTE FORCE

- Convert to `vector<char>` or use extra string to build result, then copy back.
- Works but wastes $O(n)$ auxiliary space for something that needs none.

Time: $O(n)$ | Space: $O(n)$

BETTER SOLUTION

- Use `std::reverse` / STL helpers directly — correct but hides the mechanics interviewer wants to see.

Time: $O(n)$ | Space: $O(1)$

OPTIMAL SOLUTION

- Two-pointer in-place swap from both ends toward the middle, single pass.
- Handle even/odd length uniformly with `left < right` condition.

Time: $O(n)$ | Space: $O(1)$

C++ CODE (Optimal)

`pattern01_optimal.cpp`

```
void reverseInPlace(string &s) {
    int l = 0, r = (int)s.size() - 1;
    while (l < r) {
        swap(s[l], s[r]);
        l++; r--;
    }
}
```

COMMON MISTAKES

- Using `s.length()` inside loop condition repeatedly (fine perf-wise but shows lack of awareness) instead of caching size.
- Off-by-one on the middle index for odd-length strings.
- Forgetting strings are 0-indexed and mixing 1-indexed logic from math notation.
- Creating a new string when in-place was asked.
- Not handling empty string / single character edge case.
- Using `char - '0'` for digit conversion without checking `isdigit()` first.
- Assuming ASCII-only input when Unicode/UTF-8 is possible in real systems.
- Mutating a string while iterating over it with a range-based for loop (invalidates expectations).

INTERVIEW TIPS

- State time/space complexity before writing a single line of code.
- Say out loud: “I’ll use two pointers to avoid extra space” — signals intent early.
- Mention `const string&` for read-only traversal to show pass-by-reference awareness.
- Always ask about character set (ASCII vs Unicode) before assuming.

FOLLOW-UP QUESTIONS

- Can you do it recursively? What’s the space cost then (call stack)?
- What if the string is a stream and you can’t index backward?
- How would this change for a linked-list-of-characters instead of an array?

VARIATIONS

- Easy: Reverse String (LeetCode 344)
- Medium: Reverse Words in a String
- Hard: Reverse String with k-group constraints (like Reverse Nodes in k-Group, string version)

PATTERN SUMMARY

Traversal = the foundation. Master two-pointer in-place movement, ASCII math, and precise loop bounds — every later pattern assumes this is automatic.

PATTERN 02

Frequency Count

FAANG Frequency	Very High
LeetCode Importance	Very High
Codeforces Importance	Medium
Companies Asking	Amazon, Google, Meta, Microsoft, Flipkart, Adobe
Difficulty	Easy
Time Complexity	$O(n)$
Space Complexity	$O(1)$ for fixed alphabet
Problems Covered	First Unique Character, Valid Anagram, Majority Character, Char Frequency Comparisons

INTUITION

- Almost every string problem reduces to “how many times does X appear” — a fixed-size array `int freq[26]` is the fastest possible answer for lowercase-only input.
- This pattern is the gateway to hashing, anagrams, and sliding window — interviewers use it to gauge if you reach for the right data structure by default.

PATTERN RECOGNITION — Trigger Words

- anagram
- permutation of characters
- most/least frequent character
- two strings same characters

If interviewer says “check if two strings have the same characters” — think frequency array comparison, not sorting (unless they ask for sorted output).

DRY RUN

```
s = "aabbbc"
freq['a']=2, freq['b']=3, freq['c']=1
Single pass builds the full histogram in  $O(n)$ .
```

BRUTE FORCE

- For each character, scan the whole string counting occurrences — $O(n)$ work repeated n times.

Time: $O(n^2)$ | Space: $O(1)$

BETTER SOLUTION

- Use `unordered_map<char,int>` for a general alphabet — correct but has hashing overhead vs a plain array.

Time: $O(n)$ | Space: $O(k)$

OPTIMAL SOLUTION

- `int freq[26] = {0};` increment on `c - 'a'` in one pass.
- Compare/query the array in $O(26) = O(1)$ afterward.

Time: $O(n)$ | Space: $O(1)$

C++ CODE (Optimal)

`pattern02_optimal.cpp`

```
vector<int> charFrequency(const string &s) {  
    vector<int> freq(26, 0);  
    for (char c : s) freq[c - 'a']++;  
    return freq;  
}
```

COMMON MISTAKES

- Using `unordered_map` when the alphabet is known to be fixed (unnecessary overhead).
- Forgetting to reset the frequency array between test cases in competitive programming (classic CF bug).
- Not handling uppercase vs lowercase consistently.
- Off-by-one: `c - 'a'` vs `c - 'A'` mismatch causing negative index.
- Comparing two frequency arrays with `==` on vectors when sizes differ.
- Ignoring non-alphabetic characters (spaces, digits) silently corrupting the index.
- Using `map<char,int>` ($\log n$) when `unordered_map` or array would do.
- Forgetting frequency count needs a full second pass if you're validating something, not just building the histogram.

INTERVIEW TIPS

- Say: "since constraints say lowercase English letters, I'll use a fixed array for $O(1)$ space" — shows you read constraints.
- Mention the array-vs-hashmap tradeoff explicitly; it signals CS fundamentals.
- For Unicode, pivot to `unordered_map<char32_t,int>` and say so.
- Always clarify case-sensitivity assumptions up front.

FOLLOW-UP QUESTIONS

- What if the alphabet were all Unicode? How does your approach change?
- Can you do this with $O(1)$ extra space if modification of input is allowed?
- How would you extend this to streaming data (characters arriving one at a time)?

VARIATIONS

- Easy: Valid Anagram (LeetCode 242)
- Medium: First Unique Character in a String
- Hard: Group Shifted Strings using frequency-delta encoding

PATTERN SUMMARY

Frequency array is the cheapest, fastest tool in your string toolbox. Default to `int freq[26]` for fixed alphabets — escalate to hashmap only when needed.

PATTERN 03

Palindrome Problems

FAANG Frequency	Very High
LeetCode Importance	Very High
Codeforces Importance	High
Companies Asking	Amazon, Google, Meta, Microsoft, Bloomberg, Adobe
Difficulty	Easy-Medium
Time Complexity	$O(n)$ to $O(n^2)$ depending on variant
Space Complexity	$O(1)$ to $O(n)$
Problems Covered	Valid Palindrome, Longest Palindromic Substring, Palindrome Partitioning

INTUITION

- A palindrome reads the same forwards and backwards — which means the problem is symmetric around a center (or centers, for even length).
- The center-expansion insight is what separates candidates who “get” palindromes from those who brute-force check every substring.

PATTERN RECOGNITION — Trigger Words

- reads the same backward
- symmetric string
- longest palindromic ...
- can be rearranged into a palindrome

If interviewer says “find the longest palindromic substring” — think expand-around-center or DP table, not checking every substring naively.

DRY RUN

```
s = "babad"
center i=1 ('a'): expand -> "bab" (len 3)
center i=2 ('b') even/odd checks -> "aba" also len 3
Track the max seen.
```

BRUTE FORCE

- Check every substring $O(n^2)$ of them for palindrome property in $O(n)$ each.

Time: $O(n^3)$ | Space: $O(1)$

BETTER SOLUTION

- DP table $dp[i][j]$ = true if $s[i..j]$ is palindrome, built from smaller subproblems.

Time: $O(n^2)$ | Space: $O(n^2)$

OPTIMAL SOLUTION

- Expand around every center ($2n-1$ centers for odd/even) in $O(n)$ each in the worst case.
- For true $O(n)$, use Manacher's Algorithm (Pattern 17).

Time: $O(n^2)$ expand-center / $O(n)$ Manacher | Space: $O(1)$ / $O(n)$

C++ CODE (Optimal)

pattern03_optimal.cpp

```
pair<int,int> expand(const string &s, int l, int r) {
    while (l >= 0 && r < (int)s.size() && s[l] == s[r]) { l--; r++; }
    return {l + 1, r - 1};
}

string longestPalindrome(string s) {
    int start = 0, maxLen = 0;
    for (int i = 0; i < (int)s.size(); i++) {
        auto [l1, r1] = expand(s, i, i);
        auto [l2, r2] = expand(s, i, i + 1);
        if (r1 - l1 + 1 > maxLen) { maxLen = r1 - l1 + 1; start = l1; }
        if (r2 - l2 + 1 > maxLen) { maxLen = r2 - l2 + 1; start = l2; }
    }
    return s.substr(start, maxLen);
}
```

COMMON MISTAKES

- Forgetting the even-length center case (two-character center, not one).
- Off-by-one when converting expand-around-center bounds back to substring indices.
- Using recursion for DP palindrome check without memoization, causing exponential blowup.
- Not clarifying if “palindrome” ignores case/punctuation (Valid Palindrome variant).
- Confusing 'longest palindromic substring' (contiguous) with 'subsequence' (not contiguous).
- Recomputing substring checks instead of reusing the DP table.
- Ignoring that a single character is trivially a palindrome (base case bug).
- Not considering Manacher's when interviewer explicitly asks for $O(n)$.

INTERVIEW TIPS

- Immediately clarify: substring (contiguous) vs subsequence (not contiguous) — huge algorithmic difference.
- Mention Manacher's by name even if you implement expand-around-center — shows depth.
- Draw the center-expansion visually; interviewers love the whiteboard clarity.
- State clearly why brute force $O(n^3)$ fails at scale before optimizing.

FOLLOW-UP QUESTIONS

- Can you solve it in $O(n)$ time? (Manacher's)
- What if we need to count all palindromic substrings, not just the longest?
- How would you find the longest palindromic SUBSEQUENCE instead?

VARIATIONS

- Easy: Valid Palindrome (LeetCode 125)
- Medium: Longest Palindromic Substring
- Hard: Palindrome Partitioning II (min cuts)

PATTERN SUMMARY

Palindromes are symmetry problems. Expand-around-center is the interview-safe $O(n^2)$; Manacher's is the flex $O(n)$ answer when pushed.

PATTERN 04

Two Pointer

FAANG Frequency	Very High
LeetCode Importance	Very High
Codeforces Importance	High
Companies Asking	Amazon, Google, Meta, Microsoft, Uber, Airbnb
Difficulty	Easy-Medium
Time Complexity	$O(n)$
Space Complexity	$O(1)$
Problems Covered	Valid Palindrome, Reverse Vowels, Is Subsequence, Two Sum on sorted char array

INTUITION

- Two pointers exploit monotonic structure — moving one or both pointers only forward guarantees you never redo work, collapsing an $O(n^2)$ scan into $O(n)$.
- It's the natural next step after basic traversal: instead of one index, you reason about the relationship between two moving indices.

PATTERN RECOGNITION — Trigger Words

- from both ends
- subsequence of
- without extra space
- in-place comparison

If interviewer says "check if s is a subsequence of t without extra space" — think two pointers advancing independently, not recursion.

DRY RUN

```
s="abc", t="ahbgdc"
i=0,j=0: a==a -> i=1,j=1
i=1,j=2: b==b -> i=2,j=3
i=2,j=5: c==c -> i=3 (done, s exhausted) -> true
```

BRUTE FORCE

- Generate all subsequences of t and check membership — exponential and absurd for this problem.

Time: $O(2^n)$ | Space: $O(n)$

BETTER SOLUTION

- DP table for general subsequence counting when the question asks 'how many ways', overkill for simple existence check.

Time: $O(n*m)$ | Space: $O(n*m)$

OPTIMAL SOLUTION

- Advance pointer i in s only when a match is found while j always advances through t .
- Terminates the moment i reaches $s.size()$ (success) or j exhausts t (failure).

Time: $O(n)$ | Space: $O(1)$

C++ CODE (Optimal)

pattern04_optimal.cpp

```
bool isSubsequence(const string &s, const string &t) {
    int i = 0;
    for (int j = 0; j < (int)t.size() && i < (int)s.size(); j++) {
        if (s[i] == t[j]) i++;
    }
    return i == (int)s.size();
}
```

COMMON MISTAKES

- Advancing both pointers unconditionally instead of only advancing the 'needle' pointer on match.
- Forgetting to guard array bounds before dereferencing ($i < s.size()$ check).
- Using two pointers for a problem that actually requires sliding window (different pattern, different pointer semantics).
- Not handling empty string s (trivially a subsequence of anything).
- Assuming two-pointer always means opposite ends — sometimes both start at 0 and move same direction.
- Mutating the string mid-traversal, breaking pointer assumptions.
- Missing the early-exit optimization once $i == s.size()$.
- Confusing 'subsequence' (skip allowed) with 'substring' (contiguous) requirements.

INTERVIEW TIPS

- Explicitly name the two-pointer variant: opposite-ends vs same-direction — shows structured thinking.
- Explain WHY it's $O(n)$: each pointer moves forward at most n times total, never backtracks.
- For 'k strings follow this pattern' scale up to multiple pointers and mention it proactively.
- Draw pointer positions on the whiteboard at each step for the dry run.

FOLLOW-UP QUESTIONS

- What if you had to find this for a huge number of (s, t) query pairs? (Precompute next-occurrence table)
- How would you handle Unicode/multi-byte characters?
- Can this be parallelized?

VARIATIONS

- Easy: Is Subsequence (LeetCode 392)
- Medium: Reverse Vowels of a String
- Hard: Number of Matching Subsequences (batch queries variant)

PATTERN SUMMARY

Two pointer is 'smart brute force' — it earns $O(n)$ by proving each pointer never needs to revisit a position.

PATTERN 05

Sliding Window

FAANG Frequency	Extremely High
LeetCode Importance	Extremely High
Codeforces Importance	High
Companies Asking	Amazon, Google, Meta, Microsoft, Bloomberg, Uber, Netflix
Difficulty	Medium
Time Complexity	$O(n)$
Space Complexity	$O(k)$ for window state
Problems Covered	Longest Substring Without Repeating Characters, Minimum Window Substring, Longest Repeating Char Replacement

INTUITION

- When a problem asks for the 'longest/shortest/best contiguous substring satisfying condition X', shrinking and growing a window is almost always the $O(n)$ answer.
- The window only grows via right pointer and only shrinks via left pointer — both move forward monotonically, giving amortized $O(n)$ total movement.

PATTERN RECOGNITION — Trigger Words

- longest substring with...
- shortest substring containing...
- at most k distinct
- contiguous subarray/substring satisfying a constraint

If interviewer says "longest substring without repeating characters" — think variable-size sliding window with a hashset/array tracking the window's contents.

DRY RUN

```
s="abcabcbb"
right=0..3 window="abca" -> dup 'a' -> shrink left until valid: "bca"
maxLen tracked as window grows/shrinks -> answer 3 ("abc")
```

BRUTE FORCE

- Check every substring for the validity condition — $O(n^2)$ substrings, $O(n)$ check each.

Time: $O(n^3)$ | Space: $O(n)$

BETTER SOLUTION

- Fix left, extend right until invalid, record length, move left by 1 — redundant re-scanning of overlapping ranges.

Time: $O(n^2)$ | Space: $O(k)$

OPTIMAL SOLUTION

- Maintain a window [left, right] and a frequency/state structure; expand right always, shrink left only when the window becomes invalid.
- Each character is added once and removed at most once — true $O(n)$.

Time: $O(n)$ | Space: $O(k)$

C++ CODE (Optimal)

pattern05_optimal.cpp

```
int lengthOfLongestSubstring(const string &s) {
    vector<int> lastSeen(256, -1);
    int left = 0, best = 0;
    for (int right = 0; right < (int)s.size(); right++) {
        if (lastSeen[(unsigned char)s[right]] >= left) {
            left = lastSeen[(unsigned char)s[right]] + 1;
        }
        lastSeen[(unsigned char)s[right]] = right;
        best = max(best, right - left + 1);
    }
    return best;
}
```

COMMON MISTAKES

- Shrinking the window character-by-character when a direct jump (using last-seen index) would be $O(n)$ instead of $O(n*k)$.
- Forgetting to update the frequency/state structure when a character LEAVES the window on shrink.
- Off-by-one in window length calculation: $right - left + 1$, not $right - left$.
- Using a fixed-size window loop for a variable-size window problem (wrong template entirely).
- Not resetting window state between separate sliding-window subproblems.
- Confusing 'at most k ' with 'exactly k ' — exactly k is usually solved as $atMost(k) - atMost(k-1)$.
- Using `unordered_map` when a fixed array (256 or 26) would be faster and simpler.
- Not handling the case where the whole string is the answer (never shrinks).

INTERVIEW TIPS

- Explicitly state the sliding window invariant out loud: "window $[left, right]$ is always valid/invalid because..."
- Mention amortized analysis: 'left and right each move at most n times total, so it's $O(n)$ not $O(n^2)$ '.
- For 'exactly K ' variants, mention the $atMost(K) - atMost(K-1)$ trick proactively — huge signal of experience.
- Use a whiteboard table showing window bounds evolving frame by frame.

FOLLOW-UP QUESTIONS

- How would you solve 'exactly K distinct characters' using this same window?
- What if the window needs to track a multiset of required characters (Minimum Window Substring)?
- Can you parallelize sliding window across chunks of a huge string?

VARIATIONS

- Easy: Longest Substring Without Repeating Characters (LeetCode 3)
- Medium: Longest Repeating Character Replacement
- Hard: Minimum Window Substring (LeetCode 76)

PATTERN SUMMARY

Sliding window turns 'check every substring' into a single linear pass by proving each pointer moves forward at most n times — the single most FAANG-tested string pattern.

PATTERN 06

Hashing

FAANG Frequency	Very High
LeetCode Importance	Very High
Codeforces Importance	Very High
Companies Asking	Google, Amazon, Meta, Microsoft, Databricks
Difficulty	Easy-Medium
Time Complexity	$O(n)$ average
Space Complexity	$O(n)$
Problems Covered	Group Anagrams, Two Sum variants on strings, Duplicate detection

INTUITION

- Hashing trades space for $O(1)$ average lookup — whenever you need to ask 'have I seen this before?' repeatedly, a hash-based structure turns $O(n)$ repeated scans into $O(1)$ checks.
- For strings specifically, you either hash the whole string (`unordered_map<string,...>`) or hash a signature derived from it (sorted string, frequency vector) to group equivalent strings.

PATTERN RECOGNITION — Trigger Words

- group by...
- have I seen this substring before
- duplicate detection
- anagram grouping

If interviewer says “group words that are anagrams of each other” — think hashmap keyed by a canonical signature (sorted string or frequency tuple).

DRY RUN

```
words=["eat","tea","tan"]
sorted("eat")="aet" -> bucket["aet"] += "eat"
sorted("tea")="aet" -> bucket["aet"] += "tea"
sorted("tan")="ant" -> new bucket
```

BRUTE FORCE

- Compare every pair of strings directly for the equivalence relation — $O(n^2)$ comparisons, each $O(k)$.

Time: $O(n^2 * k)$ | Space: $O(1)$

BETTER SOLUTION

- Sort each string to get a canonical key, use that as hashmap key — sorting costs $O(k \log k)$ per string.

Time: $O(n * k \log k)$ | Space: $O(n*k)$

OPTIMAL SOLUTION

- Use a frequency-vector (26 ints) as the canonical key instead of sorting — avoids the $\log k$ factor entirely.

Time: $O(n * k)$ | Space: $O(n*k)$

C++ CODE (Optimal)

`pattern06_optimal.cpp`

```
vector<vector<string>> groupAnagrams(vector<string>& strs) {
    unordered_map<string, vector<string>> buckets;
    for (auto &w : strs) {
        array<int, 26> freq{};
        for (char c : w) freq[c - 'a']++;
        string key(freq.begin(), freq.end()); // encode counts as key
        buckets[key].push_back(w);
    }
    vector<vector<string>> result;
    for (auto &[k, v] : buckets) result.push_back(v);
    return result;
}
```

COMMON MISTAKES

- Using sort-based keys when frequency-vector keys would avoid the $\log k$ factor.
- Forgetting hash collisions are possible with custom string hashing — always verify equality, don't trust hash alone.
- Using `unordered_map<string,...>` naively when the key generation itself is the bottleneck.
- Not considering case sensitivity when building canonical keys.
- Rehashing the same string multiple times instead of caching its signature.
- Ignoring load factor / rehashing cost of `unordered_map` for very large inputs (can degrade to $O(n)$ per op).
- Using a `vector<int>` directly as key without converting to a hashable type like string or tuple.
- Assuming hashmap iteration order is deterministic (it isn't in C++ `unordered_map`).

INTERVIEW TIPS

- Say explicitly: 'I'll use a canonical signature (frequency vector) as the hashmap key to avoid the $O(k \log k)$ sort cost.'
- Mention hash collision handling as a signal of understanding hashmap internals.
- If asked about custom hash functions, mention combining hashes with a good mixing function (like `boost::hash_combine`).
- Clarify whether output order matters — `unordered_map` won't preserve insertion order.

FOLLOW-UP QUESTIONS

- How would you design this for billions of strings that don't fit in memory?
- What if two different strings could produce the same hash (collision) — how do you handle it safely?
- Can you do this using a Trie instead of a hashmap?

VARIATIONS

- Easy: Contains Duplicate (string version)
- Medium: Group Anagrams (LeetCode 49)
- Hard: Group Shifted Strings (canonical delta-encoding)

PATTERN SUMMARY

Hashing is the universal 'have I seen this' tool. For strings, the real skill is choosing a cheap canonical signature, not just calling `unordered_map`.

PATTERN 07

Sorting Strings

FAANG Frequency	Medium
LeetCode Importance	Medium
Codeforces Importance	Medium
Companies Asking	Amazon, Google, Adobe, Goldman Sachs
Difficulty	Easy-Medium
Time Complexity	$O(n \log n)$
Space Complexity	$O(n)$
Problems Covered	Sort Characters By Frequency, Largest Number formed by Strings, Custom Alphabetical Order

INTUITION

- Sorting strings usually means one of two things: sorting the CHARACTERS within a string, or sorting an ARRAY of strings by a custom comparator — interviewers test if you pick the right comparator logic.
- Custom comparators for strings (e.g., 'which concatenation order gives the largest number') are a classic way to test greedy + comparator reasoning together.

PATTERN RECOGNITION — Trigger Words

- sort characters by frequency
- custom order / alien dictionary order
- arrange to form the largest/smallest number

If interviewer says "arrange these strings to form the largest possible number" — think custom comparator ($a+b$ vs $b+a$), not numeric sort.

DRY RUN

```
nums=["3","30","34","5","9"]
compare("3","30"): "330" vs "303" -> "330" bigger -> "3" before "30"
Final sorted order maximizes concatenation.
```

BRUTE FORCE

- Try every permutation of the array and evaluate the resulting concatenation — factorial blowup.

Time: $O(n! * k)$ | Space: $O(n)$

BETTER SOLUTION

- Sort by string value directly (lexicographic) — fails because "9" > "30" lexicographically doesn't match the needed order for concatenation.

Time: $O(n \log n)$ | Space: $O(n)$

OPTIMAL SOLUTION

- Sort using a custom comparator that compares $a+b$ vs $b+a$ directly — this comparator is provably transitive for this problem.

Time: $O(n \log n * k)$ | Space: $O(n)$

C++ CODE (Optimal)

pattern07_optimal.cpp

```
string largestNumber(vector<string>& nums) {
    sort(nums.begin(), nums.end(), [](const string &a, const string &b) {
        return a + b > b + a;
    });
    if (!nums.empty() && nums[0] == "0") return "0"; // all zeros edge case
    string result;
    for (auto &s : nums) result += s;
    return result;
}
```

COMMON MISTAKES

- Sorting numerically or lexicographically instead of using the $a+b$ vs $b+a$ comparator.
- Forgetting the all-zero edge case ("0","0") which should output "0", not "00".
- Using a non-transitive comparator that crashes `std::sort` in debug mode (UB in release).
- Ignoring stability requirements when duplicates with equal comparator results exist.
- Sorting characters when the problem actually wants sorting of string elements in an array (or vice versa).
- Not considering counting sort ($O(n+k)$) when the alphabet size is small and fixed.
- Assuming default string comparison operators are enough for domain-specific ordering (like 'alien dictionary').
- Forgetting to convert case before sorting when the problem is case-insensitive.

INTERVIEW TIPS

- Explicitly prove why $a+b > b+a$ is a valid transitive comparator when asked — shows rigor.
- Mention counting sort for $O(n+k)$ when only sorting fixed-alphabet characters within one string.
- Call out edge cases (all zeros, empty strings) before coding.
- State clearly whether you're sorting the string's characters or an array of strings.

FOLLOW-UP QUESTIONS

- Can you prove the comparator $a+b > b+a$ is transitive?
- How would this change for sorting by a custom alien alphabet order?
- What's the fastest way to sort characters within a single string of fixed alphabet size?

VARIATIONS

- Easy: Sort Characters By Frequency (LeetCode 451)
- Medium: Largest Number (LeetCode 179)
- Hard: Verifying an Alien Dictionary + custom total order sort

PATTERN SUMMARY

String sorting problems hinge on choosing the right comparator or encoding — rarely is default lexicographic sort actually correct.

PATTERN 08

Anagram

FAANG Frequency	Very High
LeetCode Importance	Very High
Codeforces Importance	Medium
Companies Asking	Amazon, Meta, Google, Microsoft, Flipkart
Difficulty	Easy-Medium
Time Complexity	$O(n)$
Space Complexity	$O(1)$ fixed alphabet
Problems Covered	Valid Anagram, Group Anagrams, Find All Anagrams in a String

INTUITION

- Two strings are anagrams iff their character frequency multisets are identical — this collapses the problem from 'compare all permutations' to 'compare two frequency arrays'.
- When the question involves a SLIDING window of anagrams inside a bigger string, this pattern combines directly with Sliding Window (Pattern 05).

PATTERN RECOGNITION — Trigger Words

- anagram of
- rearrangement of characters
- same characters different order
- find all starting indices of anagram

If interviewer says "find all starting indices where an anagram of p occurs in s " — think fixed-size sliding window + frequency array comparison.

DRY RUN

```
s="cbaebabacd", p="abc"
window[0..2]="cba" freq matches p's freq -> index 0 is valid
slide window by 1, update freq incrementally, compare again.
```

BRUTE FORCE

- Sort both strings and compare — correct but pays an unnecessary $O(k \log k)$ cost per comparison.

Time: $O(n * k \log k)$ | Space: $O(k)$

BETTER SOLUTION

- Build frequency arrays for both and compare arrays directly — avoids sorting cost.

Time: $O(n * k)$ | Space: $O(k)$

OPTIMAL SOLUTION

- For the sliding-window variant, maintain a running frequency diff array and a 'matches' counter updated in $O(1)$ per slide instead of comparing the whole array each time.

Time: $O(n)$ | Space: $O(1)$ fixed alphabet

C++ CODE (Optimal)

pattern08_optimal.cpp

```
vector<int> findAnagrams(string s, string p) {
    if (s.size() < p.size()) return {};
    vector<int> need(26, 0), window(26, 0), result;
    for (char c : p) need[c - 'a']++;
    int k = p.size();
    for (int i = 0; i < (int)s.size(); i++) {
        window[s[i] - 'a']++;
        if (i >= k) window[s[i - k] - 'a']--;
        if (i >= k - 1 && window == need) result.push_back(i - k + 1);
    }
    return result;
}
```

COMMON MISTAKES

- Recomputing the full frequency array from scratch on every window slide instead of incrementally updating.
- Comparing arrays with `==` in a hot loop when a running 'matched count' would be $O(1)$ per step.
- Off-by-one on window start/end indices when sliding.
- Sorting strings for a single anagram check when array comparison is cheaper.
- Forgetting `p` could be longer than `s` (immediate empty-result edge case).
- Not clearing/removing the outgoing character's count when the window slides forward.
- Assuming input is lowercase-only without confirming constraints.
- Using `unordered_map` when array indexing is faster and simpler for a-z only.

INTERVIEW TIPS

- Say: 'this is Sliding Window + frequency array, since anagram = fixed-size window with matching character counts.'
- Explicitly mention the $O(1)$ 'matches' counter trick to avoid full array comparison on each slide — big signal.
- Clarify constraints (alphabet, case-sensitivity) before coding.
- Draw the sliding window diagram with counts updating live.

FOLLOW-UP QUESTIONS

- How would you extend this to find anagram substrings across a huge text file streamed in chunks?
- What if 'anagram' allowed one wildcard character?
- Can you solve Group Anagrams using this same frequency-signature idea?

VARIATIONS

- Easy: Valid Anagram (LeetCode 242)
- Medium: Find All Anagrams in a String (LeetCode 438)
- Hard: Permutation in String combined with minimum window constraints

PATTERN SUMMARY

Anagram problems are frequency-equality problems. Combine with sliding window for substring search variants, and always prefer array over sort.

PATTERN 09

Character Mapping

FAANG Frequency	Medium
LeetCode Importance	Medium
Codeforces Importance	Medium
Companies Asking	Google, Amazon, Adobe, LinkedIn
Difficulty	Easy-Medium
Time Complexity	$O(n)$
Space Complexity	$O(1)$ fixed alphabet
Problems Covered	Isomorphic Strings, Word Pattern, Buddy Strings

INTUITION

- Character mapping problems ask you to verify a BIJECTION between two alphabets (or a string and a pattern) — you need two maps (forward and backward) to guarantee true one-to-one correspondence, not just one-directional matching.
- Forgetting the backward map is the single most common bug — it lets many-to-one mappings slip through as false positives.

PATTERN RECOGNITION — Trigger Words

- isomorphic
- one-to-one mapping
- each character maps to exactly one other character
- pattern matches word structure

If interviewer says "check if two strings are isomorphic" — think two hashmaps ($s \rightarrow t$ and $t \rightarrow s$), not just one.

DRY RUN

```
s="egg", t="add"
e->a, g->d (first g)
g->d again -> consistent
reverse: a->e, d->g consistent both ways -> true
```

BRUTE FORCE

- Try to construct explicit substitution and validate by rebuilding the string character by character with nested checks — easy to get wrong without formalizing as a map.

Time: $O(n^2)$ | Space: $O(1)$

BETTER SOLUTION

- Single hashmap $s \rightarrow t$ only — passes many test cases but fails on many-to-one mappings like $s='ab'$, $t='aa'$.

Time: $O(n)$ | Space: $O(k)$

OPTIMAL SOLUTION

- Two hashmaps (or two fixed-size arrays of 256) enforcing bijection both ways in a single pass.

Time: $O(n)$ | Space: $O(1)$ fixed 256-array

C++ CODE (Optimal)

`pattern09_optimal.cpp`

```
bool isIsomorphic(string s, string t) {
    if (s.size() != t.size()) return false;
    vector<int> mapST(256, -1), mapTS(256, -1);
    for (int i = 0; i < (int)s.size(); i++) {
        unsigned char a = s[i], b = t[i];
        if (mapST[a] == -1 && mapTS[b] == -1) {
            mapST[a] = b; mapTS[b] = a;
        } else if (mapST[a] != b || mapTS[b] != a) {
            return false;
        }
    }
    return true;
}
```

COMMON MISTAKES

- Using only a forward map, missing the many-to-one bijection violation.
- Not checking string lengths match before starting the mapping pass.
- Using `unordered_map<char,char>` when a fixed 256-array is faster and simpler.
- Forgetting to initialize map entries as 'unset' (using 0 as sentinel when 0 could be a valid mapped value).
- Mixing up direction of the two maps (s->t vs t->s) leading to logic bugs.
- Not handling Unicode/multi-byte characters if input isn't guaranteed ASCII.
- Assuming isomorphic implies same character SET size — it doesn't need distinct chars, just consistent mapping.
- Overwriting an existing mapping without checking consistency first.

INTERVIEW TIPS

- Say explicitly: 'isomorphism requires bijection, so I need forward AND backward maps.'
- Use -1 or a sentinel struct, never 0, to represent 'unmapped' when values could legitimately be 0.
- Mention Word Pattern as the 'array of words' version of this same exact idea.
- Draw the two-map dry run visually to avoid confusion.

FOLLOW-UP QUESTIONS

- How would you solve Word Pattern using the same two-map idea but on word tokens instead of characters?
- What if character mapping needed to also satisfy sorted order constraints?
- Can you do this with $O(1)$ extra space if input strings are mutable buffers?

VARIATIONS

- Easy: Word Pattern (LeetCode 290)
- Medium: Isomorphic Strings (LeetCode 205)
- Hard: Buddy Strings with swap validation

PATTERN SUMMARY

Character mapping = bijection verification. Always use two maps (forward and backward) to catch many-to-one violations.

PATTERN 10

Prefix Frequency

FAANG Frequency	Medium
LeetCode Importance	Medium
Codeforces Importance	High
Companies Asking	Google, Amazon, Adobe, Bloomberg
Difficulty	Medium
Time Complexity	$O(n)$
Space Complexity	$O(n)$
Problems Covered	Longest Common Prefix, Prefix counting with Trie, Count Prefix/Suffix pairs

INTUITION

- Prefix-based problems ask about properties of the FIRST k characters of strings — precomputing prefix frequency or prefix hash tables lets you answer prefix queries in $O(1)$ instead of recomputing from scratch each time.
- This is the conceptual bridge from simple frequency counting toward Tries (Pattern 18) and Prefix Sum on hashed strings (Pattern 12).

PATTERN RECOGNITION — Trigger Words

- longest common prefix
- how many strings start with...
- prefix matches
- shared beginning of strings

If interviewer says "find the longest common prefix among an array of strings" — think vertical character-by-character scanning or a Trie, not pairwise LCP comparison.

DRY RUN

```

strs=["flower","flow","flight"]
col0: f,f,f match
col1: l,l,l match
col2: o,o,i mismatch -> stop, answer="fl"

```

BRUTE FORCE

- Compare every pair of strings for their pairwise LCP and take the minimum — $O(n^2)$ pairs.

Time: $O(n^2 * k)$ | Space: $O(1)$

BETTER SOLUTION

- Horizontal scanning: reduce a running prefix by comparing it against each next string one at a time.

Time: $O(n*k)$ | Space: $O(1)$

OPTIMAL SOLUTION

- Vertical scanning: check column-by-column across ALL strings simultaneously, stop at first mismatch — same complexity but better early-exit and lends itself to a Trie for repeated queries.

Time: $O(n*k)$ worst, early-exit typical | Space: $O(1)$ ($O(n*k)$ if using a Trie)

C++ CODE (Optimal)

pattern10_optimal.cpp

```
string longestCommonPrefix(vector<string>& strs) {
    if (strs.empty()) return "";
    for (int col = 0; col < (int)strs[0].size(); col++) {
        char c = strs[0][col];
        for (auto &s : strs) {
            if (col >= (int)s.size() || s[col] != c) return strs[0].substr(0, col);
        }
    }
    return strs[0];
}
```

COMMON MISTAKES

- Not guarding col against shorter strings, causing out-of-bounds access.
- Comparing pairwise LCPs when vertical scanning is both simpler and equally efficient.
- Forgetting the empty array edge case entirely.
- Using a Trie when the array is queried only once (unnecessary construction overhead).
- Not early-exiting the outer loop the moment ANY mismatch is found in a column.
- Assuming all strings are the same length.
- Recomputing prefix from scratch for every string instead of shrinking a running candidate.
- Ignoring that binary-search-on-answer-length is a valid alternative when strings are very long.

INTERVIEW TIPS

- Mention both horizontal and vertical scanning approaches, then justify picking vertical for early-exit behavior.
- For REPEATED prefix queries across many calls, proactively suggest building a Trie once and querying $O(k)$ per call.
- State the edge case (empty array, single string) explicitly before coding.
- If strings are very long, mention binary search on prefix-length as an alternative.

FOLLOW-UP QUESTIONS

- What if you need to answer 1000 prefix queries on the same array — how does your approach change? (Trie)
- How would binary search on prefix length work here?
- Can you extend this to find longest common SUFFIX instead?

VARIATIONS

- Easy: Longest Common Prefix (LeetCode 14)
- Medium: Count Prefix and Suffix pairs using Trie
- Hard: Number of Distinct Prefixes across a huge string set (Trie node counting)

PATTERN SUMMARY

Prefix frequency problems reward vertical scanning for one-off queries and Tries for repeated queries — know when to escalate.

PATTERN 11

Suffix Problems

FAANG Frequency	Medium
LeetCode Importance	Medium
Codeforces Importance	High
Companies Asking	Google, Amazon, Adobe, Bloomberg, Databricks
Difficulty	Medium-Hard
Time Complexity	$O(n \log n)$ to $O(n)$
Space Complexity	$O(n)$
Problems Covered	Longest Common Suffix, Suffix Array construction, Longest Repeated Substring

INTUITION

- Suffix problems are the mirror image of prefix problems — instead of 'what starts the same', you ask 'what ends the same' or 'how do all suffixes of a string relate to each other'.
- A Suffix Array (sorted list of all suffixes) plus an LCP (longest common prefix between adjacent suffixes) array unlocks a huge class of substring problems in $O(n \log n)$ instead of brute-forcing all $O(n^2)$ substrings.

PATTERN RECOGNITION — Trigger Words

- longest repeated substring
- all suffixes sorted
- suffix comparison
- substring that appears more than once

If interviewer says "find the longest substring that appears at least twice" — think Suffix Array + LCP array, not brute substring comparison.

DRY RUN

```
s="banana"
suffixes: banana, anana, nana, ana, na, a
sorted: a, ana, anana, banana, na, nana
LCP between adjacent sorted suffixes reveals repeated substrings.
```

BRUTE FORCE

- Generate all $O(n^2)$ substrings, use a hashset to find duplicates — memory and time heavy.

Time: $O(n^3)$ or $O(n^2)$ with hashing | Space: $O(n^2)$

BETTER SOLUTION

- Build suffix array via `std::sort` of all suffixes directly (each comparison $O(n)$) — correct but not optimal.

Time: $O(n^2 \log n)$ | Space: $O(n^2)$

OPTIMAL SOLUTION

- Build suffix array via prefix-doubling / radix ranking in $O(n \log n)$, then compute LCP array via Kasai's algorithm in $O(n)$.

Time: $O(n \log n)$ | Space: $O(n)$

C++ CODE (Optimal)

`pattern11_optimal.cpp`

```
// Simplified  $O(n \log^2 n)$  suffix array (interview-safe, not the fastest known)
vector<int> buildSuffixArray(const string &s) {
    int n = s.size();
    vector<int> sa(n), rank_(n), tmp(n);
    iota(sa.begin(), sa.end(), 0);
    for (int i = 0; i < n; i++) rank_[i] = s[i];
    for (int k = 1; k < n; k <= 1) {
        auto cmp = [&](int a, int b) {
            if (rank_[a] != rank_[b]) return rank_[a] < rank_[b];
            int ra = a + k < n ? rank_[a + k] : -1;
            int rb = b + k < n ? rank_[b + k] : -1;
            return ra < rb;
        };
        sort(sa.begin(), sa.end(), cmp);
        tmp[sa[0]] = 0;
        for (int i = 1; i < n; i++)
            tmp[sa[i]] = tmp[sa[i-1]] + (cmp(sa[i-1], sa[i]) ? 1 : 0);
        rank_ = tmp;
    }
    return sa;
}
```

COMMON MISTAKES

- Using naive $O(n^2 \log n)$ suffix sorting when prefix-doubling $O(n \log n)$ is expected at senior level.
- Forgetting the LCP array (Kasai's algorithm) is what actually answers 'longest repeated substring', not the suffix array alone.
- Off-by-one errors in rank comparisons during doubling.
- Not handling the sentinel character (often appended, like '\$') to simplify suffix comparisons.
- Confusing Suffix Array with Suffix Tree — different structures, different complexity tradeoffs.
- Rebuilding the suffix array from scratch for every query instead of reusing it.
- Ignoring memory costs for very large strings (suffix structures can be memory-heavy).
- Not mentioning simpler alternatives (rolling hash + binary search) when full suffix array is overkill.

INTERVIEW TIPS

- For most interviews, mention rolling hash + binary search on length as a lighter-weight alternative to full suffix arrays.
- Only build a full suffix array if the interviewer explicitly wants multiple substring queries answered efficiently.
- Name-drop Kasai's algorithm for LCP array construction — shows CP-level depth.
- Clarify string vs multiple strings (generalized suffix array) scope before diving in.

FOLLOW-UP QUESTIONS

- How would you find the longest common substring between TWO different strings using suffix structures?
- What's the difference between a Suffix Array and a Suffix Tree, and when would you pick one over the other?
- Can rolling hash + binary search solve this more simply for an interview setting?

VARIATIONS

- Easy: Longest Common Suffix (simple two-pointer from the end)
- Medium: Longest Repeated Substring via Suffix Array + LCP
- Hard: Longest Common Substring across two strings using Generalized Suffix Array

PATTERN SUMMARY

Suffix problems escalate to Suffix Arrays and LCP arrays for heavy multi-query workloads — but rolling hash + binary search is often the pragmatic interview answer.

PATTERN 12

Prefix Sum on String

FAANG Frequency	Medium
LeetCode Importance	Medium
Codeforces Importance	High
Companies Asking	Google, Amazon, Microsoft, Goldman Sachs
Difficulty	Medium
Time Complexity	$O(n)$ preprocess, $O(1)$ query
Space Complexity	$O(n)$
Problems Covered	Range character frequency queries, Number of substrings with equal counts, Prefix XOR/hash on strings

INTUITION

- Just like numeric prefix sums let you answer 'sum of range $[l,r]$ ' in $O(1)$, a prefix-count array per character lets you answer 'how many times does char c appear in range $[l,r]$ ' in $O(1)$ after $O(n*26)$ preprocessing.
- This generalizes to prefix XOR-hashes for detecting repeated patterns and prefix hash arrays (Pattern 13) for substring equality checks.

PATTERN RECOGNITION — Trigger Words

- range queries on substring
- how many times does X occur between indices l and r
- multiple queries on a fixed string

If interviewer says "you'll get Q queries asking for the count of character c in range $[l,r]$ " — think prefix count arrays, $O(n*26)$ build, $O(1)$ per query.

DRY RUN

```
s="abcabc", prefix['a'] = [0,1,1,1,2,2,2]
query(l=1,r=4,'a') = prefix['a'][5] - prefix['a'][1] = 2-1 = 1
```

BRUTE FORCE

- For each query, scan the substring range directly and count — fine for few queries, terrible for many.

Time: $O(Q \cdot n)$ | Space: $O(1)$

BETTER SOLUTION

- Precompute prefix sums for only the characters actually queried, lazily — saves memory but adds bookkeeping complexity.

Time: $O(n)$ per distinct char + $O(1)$ query | Space: $O(n)$ per char

OPTIMAL SOLUTION

- Precompute a full $\text{prefix}[26][n+1]$ table once in $O(26n)$, answer every query in $O(1)$.

Time: $O(26n)$ preprocess, $O(1)$ per query | Space: $O(26n)$

C++ CODE (Optimal)

`pattern12_optimal.cpp`

```
vector<array<int,26>> buildPrefixCounts(const string &s) {
    int n = s.size();
    vector<array<int,26>> prefix(n + 1);
    prefix[0].fill(0);
    for (int i = 0; i < n; i++) {
        prefix[i + 1] = prefix[i];
        prefix[i + 1][s[i] - 'a']++;
    }
    return prefix;
}

int rangeCount(vector<array<int,26>> &prefix, int l, int r, char c) {
    return prefix[r + 1][c - 'a'] - prefix[l][c - 'a'];
}
```

COMMON MISTAKES

- Off-by-one between inclusive $[l, r]$ query semantics and the prefix array's 1-indexed offset.
- Building a full prefix table when only a handful of distinct characters are ever queried (memory waste).
- Forgetting to copy the previous row before incrementing ($\text{prefix}[i+1] = \text{prefix}[i]$ first).
- Recomputing prefix sums per query instead of once upfront.
- Confusing prefix COUNT arrays (per character) with prefix SUM arrays (numeric) — different use cases.
- Not considering prefix XOR/hash for equality-style queries instead of count-style queries.
- Ignoring memory cost for very large alphabets (Unicode) with this dense array approach.
- Using `vector<vector<int>>` instead of `array<int, 26>` for a marginal but real cache/perf hit.

INTERVIEW TIPS

- Clarify upfront how MANY queries are expected — that justifies the $O(26n)$ preprocessing cost.
- Mention the space/time tradeoff explicitly: $O(26n)$ space buys $O(1)$ query time.
- For equality-of-substrings across ranges, pivot to prefix HASHES (Pattern 13) instead of counts.
- Draw the prefix table for 2-3 characters as a worked example on the whiteboard.

FOLLOW-UP QUESTIONS

- How would you adapt this for range UPDATES as well as range queries (Fenwick Tree / BIT per character)?
- What if the alphabet were Unicode instead of just lowercase English?
- Can prefix sums help detect anagram substrings in $O(1)$ per range?

VARIATIONS

- Easy: Range character count queries on a static string
- Medium: Number of substrings with all characters having equal frequency
- Hard: Range updates + range character-frequency queries via Fenwick Trees (BIT) per character

PATTERN SUMMARY

Prefix count/sum arrays turn repeated range queries from $O(n)$ each into $O(1)$ each — the classic preprocessing-for-queries tradeoff, string edition.

PATTERN 13

Rolling Hash

FAANG Frequency	High
LeetCode Importance	Medium
Codeforces Importance	Very High
Companies Asking	Google, Amazon, Databricks, Codeforces-heavy companies
Difficulty	Medium
Time Complexity	$O(n)$ build, $O(1)$ compare
Space Complexity	$O(n)$
Problems Covered	Repeated DNA Sequences, Substring equality checks, Longest Duplicate Substring (with binary search)

INTUITION

- A rolling hash lets you compute the hash of every substring of a fixed length in $O(1)$ amortized per shift, by removing the outgoing character's contribution and adding the incoming one — avoiding recomputation from scratch.
- This is the practical, interview-friendly alternative to full suffix structures when you need 'does this substring appear elsewhere' style queries.

PATTERN RECOGNITION — Trigger Words

- substring equality check
- does this pattern repeat
- compare many substrings for equality fast

If interviewer says “find all repeated 10-character DNA substrings” — think rolling hash (Rabin-Karp style) with a hashset of seen hash values, not substring comparison.

DRY RUN

```
s="aabaaba", window size k=3
hash("aab") computed directly
hash("aba") = (hash("aab") - 'a'*base^2)*base + 'a' (rolling update)
```

BRUTE FORCE

- Extract every substring of length k as an actual string, hash or compare directly — $O(k)$ cost per substring.

Time: $O(n*k)$ | Space: $O(n*k)$

BETTER SOLUTION

- Use `std::hash<string>` per substring — correct but still pays $O(k)$ per substring since a new string is built each time.

Time: $O(n*k)$ | Space: $O(n)$

OPTIMAL SOLUTION

- Precompute base powers, roll the hash forward in $O(1)$ per shift by subtracting the leaving character and adding the entering one.
- Use double hashing (two different mod/base pairs) to avoid adversarial collisions.

Time: $O(n)$ | Space: $O(n)$

C++ CODE (Optimal)

`pattern13_optimal.cpp`

```
vector<string> findRepeatedDnaSequences(string s) {
    const int K = 10;
    unordered_map<long long, int> seen;
    vector<string> result;
    long long base = 131, mod = 1e9 + 7, hash_ = 0, power = 1;
    for (int i = 0; i < K - 1; i++) power = (power * base) % mod;
    for (int i = 0; i < (int)s.size(); i++) {
        hash_ = (hash_ * base + s[i]) % mod;
        if (i >= K) {
            hash_ = (hash_ - (s[i - K] * power) % mod * base % mod + (long long)mod*mod) % mod;
        }
        if (i >= K - 1) {
            if (++seen[hash_] == 2) result.push_back(s.substr(i - K + 1, K));
        }
    }
    return result;
}
```

COMMON MISTAKES

- Using a single hash function/modulus, making the solution vulnerable to adversarial collisions (common CF anti-hash-test trick).
- Getting negative values from modulo subtraction in C++ (must add mod back before taking % again).
- Forgetting to precompute $\text{base}^{(k-1)}$ once instead of recomputing $\text{pow}()$ per shift.
- Not verifying actual string equality after a hash match when correctness (not just approximate) matters.
- Off-by-one in the rolling window's remove-step (removing the wrong character's contribution).
- Choosing a small modulus that overflows/collides frequently for large inputs.
- Assuming a single 32-bit hash is safe — 64-bit or double hashing is the safer default in interviews and CP.
- Not clarifying whether pattern length is fixed (rolling hash) or variable (needs different technique).

INTERVIEW TIPS

- Mention double hashing (two mod/base pairs) proactively to preempt the 'what about collisions' follow-up.
- Explain the rolling update formula clearly: remove leaving char's weighted contribution, shift, add new char.
- Compare rolling hash vs KMP/Z-algorithm for exact pattern matching — know when each is preferred.
- Always verify with direct string comparison after a hash hit if 100% correctness is required (not just probabilistic).

FOLLOW-UP QUESTIONS

- How do you defend against adversarial anti-hash test cases (common in Codeforces)?
- How would you find the LONGEST duplicated substring, not just fixed-length ones? (binary search on length + rolling hash)
- What's the tradeoff between rolling hash and building a suffix automaton for this?

VARIATIONS

- Easy: Repeated DNA Sequences (LeetCode 187)
- Medium: Longest Duplicate Substring using rolling hash + binary search on length
- Hard: Distinct Substrings Count using rolling hash + hashset at every length

PATTERN SUMMARY

Rolling hash gives $O(1)$ amortized substring comparison — the pragmatic middle ground between naive comparison and full suffix structures. Always double-hash for safety.

PATTERN 14

Rabin Karp

FAANG Frequency	Medium
LeetCode Importance	Medium
Codeforces Importance	High
Companies Asking	Google, Amazon, Bloomberg, CP-focused companies
Difficulty	Medium
Time Complexity	$O(n+m)$ average
Space Complexity	$O(1)$
Problems Covered	Implement strStr(), Repeated Substring Pattern, Multi-pattern search variants

INTUITION

- Rabin-Karp is rolling hash (Pattern 13) applied specifically to pattern matching: hash the pattern once, roll the hash across the text, and only do a direct character comparison when hashes collide — giving average $O(n+m)$ time.
- It shines when you need to search for MULTIPLE patterns of the same length simultaneously, since you can hash all patterns into a set once and check the rolling text hash against that set.

PATTERN RECOGNITION — Trigger Words

- find pattern occurrences in text
- substring search
- multiple pattern matching same length

If interviewer says “search for this pattern in the text efficiently” and then adds “what if there were 1000 patterns?” — think Rabin-Karp with a hashset of pattern-hashes.

DRY RUN

```
text="abcabcabc", pattern="cab"
hash(pattern) computed once
roll hash across text windows of length 3, compare to pattern hash, verify on match
```

BRUTE FORCE

- Naive substring search: for every starting index, compare up to m characters — classic $O(n*m)$ worst case (e.g. "aaaa...a" pattern).

Time: $O(n*m)$ | Space: $O(1)$

BETTER SOLUTION

- KMP or Z-algorithm guarantee $O(n+m)$ worst case deterministically — better worst-case guarantee than Rabin-Karp.

Time: $O(n+m)$ | Space: $O(m)$

OPTIMAL SOLUTION

- Rabin-Karp for the common case (average $O(n+m)$) especially valuable for MULTI-pattern search of fixed length; fall back to KMP if adversarial worst-case guarantees are required.

Time: $O(n+m)$ average, $O(n*m)$ worst-case | Space: $O(1)$

C++ CODE (Optimal)

pattern14_optimal.cpp

```
int strStrRabinkarp(string haystack, string needle) {
    int n = haystack.size(), m = needle.size();
    if (m > n) return -1;
    long long base = 131, mod = 1e9 + 7, patHash = 0, curHash = 0, power = 1;
    for (int i = 0; i < m - 1; i++) power = (power * base) % mod;
    for (int i = 0; i < m; i++) {
        patHash = (patHash * base + needle[i]) % mod;
        curHash = (curHash * base + haystack[i]) % mod;
    }
    for (int i = 0; ; i++) {
        if (curHash == patHash && haystack.substr(i, m) == needle) return i;
        if (i + m >= n) break;
        curHash = (curHash - (haystack[i] * power) % mod * base % mod + (long long)mod * mod)
% mod;
        curHash = (curHash + haystack[i + m]) % mod;
    }
    return -1;
}
```

COMMON MISTAKES

- Skipping the direct string verification after a hash match (hash collisions do happen).
- Using a single hash/modulus vulnerable to adversarial worst-case inputs designed to force collisions.
- Handling negative modulo results incorrectly in C++ after subtraction.
- Recomputing the pattern hash inside the search loop instead of once upfront.
- Not considering KMP when the interviewer explicitly asks for guaranteed $O(n+m)$ worst case.
- Off-by-one on window boundaries when rolling the hash across the text.
- Forgetting the $\text{base}^{(m-1)}$ precomputation, recalculating it on every shift (perf bug).
- Ignoring the multi-pattern use case where Rabin-Karp actually outshines KMP (hash all patterns into a set once).

INTERVIEW TIPS

- Proactively state Rabin-Karp's average-case vs worst-case tradeoff vs KMP's guaranteed worst case.
- Mention the multi-pattern search strength explicitly — it's the scenario where Rabin-Karp is the BEST choice, not just an alternative.
- Always mention verifying on hash match to guard against collisions.
- Use double hashing if pushed on adversarial test robustness.

FOLLOW-UP QUESTIONS

- How would you search for 1000 different patterns of the same length in one pass over the text?
- What's Rabin-Karp's worst-case time complexity and when does it degrade to that?
- Compare Rabin-Karp vs KMP vs Z-algorithm — when would you pick each?

VARIATIONS

- Easy: Implement `strStr()` (LeetCode 28)
- Medium: Repeated Substring Pattern using Rabin-Karp rolling hash
- Hard: Multi-pattern substring search (multiple needles, one haystack, one pass)

PATTERN SUMMARY

Rabin-Karp is rolling hash specialized for pattern search — average-case excellent, especially for multi-pattern search; always verify hash matches directly.

PATTERN 15

KMP (Knuth-Morris-Pratt)

FAANG Frequency	High
LeetCode Importance	Medium
Codeforces Importance	Very High
Companies Asking	Google, Amazon, Meta, CP-heavy companies (Codeforces regulars)
Difficulty	Hard
Time Complexity	$O(n+m)$
Space Complexity	$O(m)$
Problems Covered	Implement strStr(), Shortest Palindrome, Repeated Substring Pattern

INTUITION

- KMP's core insight: when a mismatch happens after matching k characters, you already KNOW information about the pattern's own internal structure — the 'failure function' (LPS array) tells you exactly how far back to jump WITHOUT re-scanning the text.
- This guarantees the text pointer NEVER moves backward — giving a hard $O(n+m)$ worst-case guarantee, unlike naive search or even Rabin-Karp's average case.

PATTERN RECOGNITION — Trigger Words

- guaranteed linear time pattern matching
- exact substring search worst case matters
- self-similarity of a pattern (prefix = suffix)

If interviewer says "find pattern occurrences with guaranteed $O(n+m)$, no average-case hand-waving" — think KMP with LPS (longest proper prefix which is also suffix) array.

DRY RUN

```
pattern="ababaca"
LPS array: [0,0,1,2,3,0,1]
At index 5 ('c' vs mismatch), LPS[4]=3 tells us to resume comparing from position 3 in the pattern, not restart from 0.
```

BRUTE FORCE

- Naive matching restarts comparison from the next text index on every mismatch — $O(n*m)$ worst case on adversarial inputs like `text='aaaa...a'`, `pattern='aaaa...ab'`.

Time: $O(n*m)$ | Space: $O(1)$

BETTER SOLUTION

- Rabin-Karp gives average $O(n+m)$ but can degrade to $O(n*m)$ on adversarial hash collisions.

Time: $O(n+m)$ average | Space: $O(1)$

OPTIMAL SOLUTION

- KMP precomputes the LPS (failure) array in $O(m)$, then scans text in $O(n)$ total, NEVER backtracking the text pointer.

Time: $O(n+m)$ guaranteed | Space: $O(m)$

C++ CODE (Optimal)

`pattern15_optimal.cpp`

```
vector<int> computeLPS(const string &pat) {
    int m = pat.size();
    vector<int> lps(m, 0);
    int len = 0, i = 1;
    while (i < m) {
        if (pat[i] == pat[len]) { lps[i++] = ++len; }
        else if (len) { len = lps[len - 1]; }
        else { lps[i++] = 0; }
    }
    return lps;
}

int kmpSearch(const string &text, const string &pat) {
    vector<int> lps = computeLPS(pat);
    int i = 0, j = 0;
    while (i < (int)text.size()) {
        if (text[i] == pat[j]) { i++; j++; if (j == (int)pat.size()) return i - j; }
        else if (j) j = lps[j - 1];
        else i++;
    }
    return -1;
}
```

COMMON MISTAKES

- Confusing LPS[i] (longest proper prefix-suffix ending at i) with a simple prefix count — must be PROPER (not the whole substring itself).
- Moving the text pointer i backward on mismatch (defeats the entire point of KMP's $O(n)$ guarantee).
- Off-by-one errors building the LPS array (classic KMP bug, always dry-run on a real example).
- Forgetting the `len = lps[len-1]` fallback step, causing infinite loops or wrong jumps.
- Not resetting j to 0 (not -1 or garbage) when len is already 0 and a mismatch occurs.
- Assuming KMP is always faster in practice than Rabin-Karp or brute-force for small inputs (constant factors matter).
- Forgetting to explain the LPS array's role clearly — interviewers penalize 'magic array' hand-waving.
- Using KMP when the problem is actually about pattern STRUCTURE (like 'shortest palindrome') without recognizing the reduction.

INTERVIEW TIPS

- Build the LPS array from scratch on the whiteboard with a concrete pattern before jumping to code — interviewers want to see you understand it, not memorized.
- State explicitly: 'the text pointer never moves backward, that's the $O(n)$ guarantee' — the single most important sentence in a KMP interview.
- For 'Shortest Palindrome' style problems, mention the `s + '#' + reverse(s)` trick using KMP's LPS array — huge signal of pattern-matching maturity.
- Compare tradeoffs vs Z-algorithm (Pattern 16) — they solve overlapping problems with different array semantics.

FOLLOW-UP QUESTIONS

- How would you use KMP's LPS array to solve 'Shortest Palindrome' (LeetCode 214)?
- What's the difference between KMP's LPS array and the Z-array?
- How would you extend KMP for multiple pattern search (Aho-Corasick)?

VARIATIONS

- Easy: Implement `strStr()` (LeetCode 28) using KMP
- Medium: Repeated Substring Pattern via LPS array property
- Hard: Shortest Palindrome using KMP on `s + '#' + reverse(s)`

PATTERN SUMMARY

KMP guarantees $O(n+m)$ via the LPS failure function, ensuring the text pointer never backtracks — a must-know for any 'guaranteed linear time' pattern matching question.

PATTERN 16

Z Algorithm

FAANG Frequency	Medium
LeetCode Importance	Low-Medium
Codeforces Importance	Very High
Companies Asking	Google, CP-heavy companies, Codeforces/ICPC contexts
Difficulty	Hard
Time Complexity	$O(n)$
Space Complexity	$O(n)$
Problems Covered	Pattern matching via concatenation trick, Counting distinct patterns, String periodicity

INTUITION

- The Z-array $Z[i]$ stores the length of the longest substring starting at i that matches a PREFIX of the string — a single array that encodes an enormous amount of self-similarity information in $O(n)$.
- Concatenating pattern + separator + text (pattern#text) and computing the Z-array of the result reduces pattern matching to 'find all i where $Z[i] == \text{pattern length}$ ' — elegant and CP-favorite.

PATTERN RECOGNITION — Trigger Words

- prefix matches at every position
- string periodicity
- pattern matching via concatenation trick
- Codeforces string problem needing $O(n)$

If interviewer/contest says "for every position, how long does the match with the prefix extend" — think Z-algorithm directly, it's built for exactly this.

DRY RUN

```
s="aabxaab"
Z[0] is undefined/whole length by convention
Z[1]=1 ("a" matches prefix "a")
Z[4]=3 ("aab" matches prefix "aab")
```

BRUTE FORCE

- For every index i , directly compare characters against the prefix until mismatch — $O(n)$ per index.

Time: $O(n^2)$ | Space: $O(1)$

BETTER SOLUTION

- KMP-based pattern matching solves the SAME concatenation-trick problems but with a differently-defined array (LPS vs Z) — comparable complexity, different mental model.

Time: $O(n+m)$ | Space: $O(m)$

OPTIMAL SOLUTION

- Z-algorithm maintains a window $[L,R]$ representing the rightmost prefix-match found so far, reusing previously computed Z-values to avoid redundant comparisons.

Time: $O(n)$ | Space: $O(n)$

C++ CODE (Optimal)

pattern16_optimal.cpp

```
vector<int> zFunction(const string &s) {
    int n = s.size();
    vector<int> z(n, 0);
    int l = 0, r = 0;
    for (int i = 1; i < n; i++) {
        if (i < r) z[i] = min(r - i, z[i - l]);
        while (i + z[i] < n && s[z[i]] == s[i + z[i]]) z[i]++;
        if (i + z[i] > r) { l = i; r = i + z[i]; }
    }
    return z;
}
// Pattern matching: build zFunction(pattern + '#' + text), find Z[i] == pattern.size()
```

COMMON MISTAKES

- Forgetting to reuse the $[L,R]$ window optimization, degrading the algorithm back to $O(n^2)$.
- Off-by-one on the window boundary conditions ($i < r$ vs $i \leq r$).
- Choosing a separator character that could actually appear in the pattern or text (must be guaranteed absent).
- Confusing Z-array semantics with LPS array semantics — they answer related but distinctly different questions.
- Not handling $Z[0]$ correctly (conventionally undefined/unused, easy to misuse).
- Using Z-algorithm when KMP would be simpler for a single pattern-in-text search (both work, but interviewer may expect KMP for classic 'strStr').
- Forgetting the separator adds $O(m)$ to space/time — negligible but should be mentioned.
- Not explaining the reuse logic ($z[i] = \min(r-i, z[i-l])$) when asked — the crux of why it's $O(n)$.

INTERVIEW TIPS

- Explain the $[L,R]$ window reuse trick explicitly — that's the single insight that makes Z-algorithm $O(n)$ instead of $O(n^2)$.
- Mention the `pattern+'#'+text` concatenation trick as the standard way to repurpose Z-array for pattern matching.
- Compare directly to KMP's LPS array when asked — shows you understand BOTH major linear-time string-matching approaches.
- Pick a separator character guaranteed not to appear in either string (state this assumption explicitly).

FOLLOW-UP QUESTIONS

- How does the Z-array differ conceptually from the KMP LPS array?
- Can you use the Z-array to find the smallest period of a string?
- How would this generalize to multiple pattern search?

VARIATIONS

- Easy: Basic Z-array construction and interpretation
- Medium: Pattern matching via `pattern#text` Z-array trick
- Hard: Finding the smallest string period using Z-array properties

PATTERN SUMMARY

The Z-algorithm is CP's favorite $O(n)$ self-similarity tool — master the $[L,R]$ window reuse insight, and the `pattern#text` trick for matching.

PATTERN 17

Manacher's Algorithm

FAANG Frequency	Medium
LeetCode Importance	Low-Medium
Codeforces Importance	Very High
Companies Asking	Google, CP-heavy companies, occasionally Meta/Amazon for senior roles
Difficulty	Hard
Time Complexity	$O(n)$
Space Complexity	$O(n)$
Problems Covered	Longest Palindromic Substring in true $O(n)$, Count All Palindromic Substrings in $O(n)$

INTUITION

- Manacher's algorithm extends the center-expansion idea (Pattern 03) but REUSES previously computed palindrome radii using symmetry around a known palindromic center, avoiding redundant expansion work entirely.
- Transforming the string with separators (e.g. "aba" -> "^#a#b#a#\$") unifies even and odd length palindrome handling into one uniform algorithm.

PATTERN RECOGNITION — Trigger Words

- longest palindromic substring in $O(n)$
- count all palindromic substrings efficiently
- interviewer explicitly rejects $O(n^2)$

If interviewer says "can you do longest palindromic substring faster than $O(n^2)$?" — Manacher's is the definitive $O(n)$ answer.

DRY RUN

```
s="aba" -> transformed "^#a#b#a#$"
radius array P computed with center/right-boundary reuse
max radius maps back to original substring bounds
```

BRUTE FORCE

- Check every substring directly for palindrome property — the naive $O(n^3)$ approach.

Time: $O(n^3)$ | Space: $O(1)$

BETTER SOLUTION

- Expand-around-center (Pattern 03) achieves $O(n^2)$, which is acceptable in most interviews but not truly optimal.

Time: $O(n^2)$ | Space: $O(1)$

OPTIMAL SOLUTION

- Manacher's maintains a 'center' and 'right boundary' of the rightmost known palindrome, using mirror symmetry to initialize new expansions instead of starting from scratch.

Time: $O(n)$ | Space: $O(n)$

C++ CODE (Optimal)

pattern17_optimal.cpp

```
string longestPalindromeManacher(string s) {
    string t = "^#";
    for (char c : s) { t += c; t += '#'; }
    t += "$";
    int n = t.size();
    vector<int> p(n, 0);
    int center = 0, right = 0;
    for (int i = 1; i < n - 1; i++) {
        if (i < right) p[i] = min(right - i, p[2 * center - i]);
        while (t[i + p[i] + 1] == t[i - p[i] - 1]) p[i]++;
        if (i + p[i] > right) { center = i; right = i + p[i]; }
    }
    int maxLen = 0, centerIdx = 0;
    for (int i = 1; i < n - 1; i++) if (p[i] > maxLen) { maxLen = p[i]; centerIdx = i; }
    int start = (centerIdx - maxLen) / 2;
    return s.substr(start, maxLen);
}
```

COMMON MISTAKES

- Forgetting the sentinel characters ('^' and '\$') that avoid out-of-bounds checks during expansion.
- Not reusing the mirror value $\min(\text{right}-i, p[2*\text{center}-i])$ — without this, it degrades back to $O(n^2)$.
- Off-by-one converting the transformed-string radius back to original string start/length.
- Choosing a separator character that could collide with actual input characters.
- Not updating center/right correctly when a new palindrome extends past the current right boundary.
- Assuming Manacher's is always necessary — for most interviews, expand-around-center $O(n^2)$ is accepted; only escalate when explicitly pushed.
- Skipping the string transformation step and trying to handle odd/even cases separately (defeats the elegance and correctness of the approach).
- Miscalculating palindrome length: radius in transformed string equals length in original string (not radius/2, due to the # separators).

INTERVIEW TIPS

- Only reach for Manacher's if the interviewer explicitly asks for true $O(n)$ — leading with expand-around-center first shows good judgement about over-engineering.
- Explain the mirror-symmetry reuse trick clearly: 'if i is within the current rightmost palindrome, I can initialize using its mirror's value'.
- Mention that the transformed string with separators elegantly unifies odd/even palindrome handling.
- Be ready to explain why radius-in-transformed-string equals length-in-original-string.

FOLLOW-UP QUESTIONS

- Can you count ALL palindromic substrings in $O(n)$ using this same P array?
- How would you adapt this to find the longest palindromic substring across streaming input?
- What's the tradeoff between Manacher's and Eertree (palindromic tree) for counting distinct palindromic substrings?

VARIATIONS

- Easy: Longest Palindromic Substring via expand-around-center ($O(n^2)$, stepping stone)
- Medium: Count all palindromic substrings using Manacher's P array
- Hard: Palindromic Tree (Eertree) for counting distinct palindromic substrings

PATTERN SUMMARY

Manacher's is the $O(n)$ palindrome champion — mirror-symmetry reuse plus a transformed string unify odd/even cases into one clean linear pass.

PATTERN 18

Trie (Prefix Tree) Basics

FAANG Frequency	Very High
LeetCode Importance	High
Codeforces Importance	Medium
Companies Asking	Google, Amazon, Meta, Microsoft, Uber
Difficulty	Medium
Time Complexity	$O(k)$ insert/search (k = word length)
Space Complexity	$O(\text{total characters} * 26)$
Problems Covered	Implement Trie, Word Search II, Longest Word in Dictionary, Autocomplete systems

INTUITION

- A Trie stores strings as paths from root to leaf, sharing common prefixes across many words — turning 'does this prefix exist among N words' from $O(N*k)$ into $O(k)$, independent of how many words are stored.
- It's the natural escalation from Prefix Frequency (Pattern 10) when you need REPEATED prefix queries across a large, static or growing dictionary.

PATTERN RECOGNITION — Trigger Words

- dictionary of words
- prefix search / autocomplete
- word search on a grid with a word list
- many repeated prefix/word lookups

If interviewer says "implement autocomplete for a dictionary of 100,000 words" — think Trie, not scanning all words per keystroke.

DRY RUN

```
insert("cat"), insert("car"), insert("dog")
root -> c -> a -> t (end)
      -> r (end)
root -> d -> o -> g (end)
search("ca") as prefix -> true (path exists, no end-flag needed)
```

BRUTE FORCE

- Store all words in a list/array, scan linearly for every prefix/word query — $O(N*k)$ per query.

Time: $O(N*k)$ per query | Space: $O(N*k)$

BETTER SOLUTION

- Use a sorted array/set of words with binary search for prefix range — $O(\log N * k)$ per query, better but still not $O(k)$.

Time: $O(\log N * k)$ | Space: $O(N*k)$

OPTIMAL SOLUTION

- Build a Trie: each node has 26 children pointers and an is-end flag; insert/search cost is $O(k)$, fully independent of N (number of words stored).

Time: $O(k)$ insert/search | Space: $O(\text{total chars} * 26)$ worst case

C++ CODE (Optimal)

pattern18_optimal.cpp

```
struct TrieNode {
    TrieNode* child[26] = {};
    bool isEnd = false;
};

class Trie {
    TrieNode* root = new TrieNode();
public:
    void insert(const string &word) {
        TrieNode* node = root;
        for (char c : word) {
            int idx = c - 'a';
            if (!node->child[idx]) node->child[idx] = new TrieNode();
            node = node->child[idx];
        }
        node->isEnd = true;
    }

    bool search(const string &word) {
        TrieNode* node = traverse(word);
        return node && node->isEnd;
    }

    bool startsWith(const string &prefix) {
        return traverse(prefix) != nullptr;
    }

private:
    TrieNode* traverse(const string &s) {
        TrieNode* node = root;
        for (char c : s) {
            int idx = c - 'a';
            if (!node->child[idx]) return nullptr;
            node = node->child[idx];
        }
        return node;
    }
};
```

COMMON MISTAKES

- Confusing `search()` (exact word match, needs `isEnd` flag) with `startsWith()` (prefix existence, no `isEnd` needed).
- Memory leaks from raw `new TrieNode()` without cleanup — mention smart pointers or an arena allocator for production code.
- Using `unordered_map<char, TrieNode*>` when a fixed array `child[26]` is faster for lowercase-only alphabets.
- Not marking `isEnd` correctly, causing `search()` to return true for prefixes that were never actually inserted as full words.
- Ignoring memory overhead of 26 pointers per node for very sparse datasets (consider compressed/radix trie).
- Forgetting to handle case sensitivity/character set assumptions explicitly.
- Rebuilding the Trie from scratch for incremental inserts instead of supporting dynamic insertion.
- Not considering a Trie is overkill for a SINGLE query (better for repeated/many queries).

INTERVIEW TIPS

- State clearly: 'Trie insert/search is $O(k)$, independent of the number of words stored — that's the key win over a hashmap/list of words.'
- Explicitly distinguish `startsWith` (prefix) vs `search` (exact word) methods — interviewers test this precisely.
- Mention compressed tries (radix trees) or DAWGs as space-optimized alternatives for production-scale dictionaries.
- For grid-based word search problems, mention building the Trie from the word list FIRST, then DFS the grid pruning via Trie — classic Word Search II technique.

FOLLOW-UP QUESTIONS

- How would you support wildcard search ('.' matches any character) in a Trie?
- How would you build an autocomplete system that returns the top-k most frequent completions?
- How would you reduce memory for a Trie storing millions of sparse words (compressed trie / DAWG)?

VARIATIONS

- Easy: Implement Trie (Prefix Tree) (LeetCode 208)
- Medium: Design Add and Search Words Data Structure (wildcard Trie)
- Hard: Word Search II (LeetCode 212) — Trie + grid DFS with pruning

PATTERN SUMMARY

Trie turns prefix/word lookups into $O(k)$, independent of dictionary size — the go-to structure whenever a word list is queried repeatedly.

PATTERN 19

String Compression

FAANG Frequency	Medium
LeetCode Importance	Medium
Codeforces Importance	Medium
Companies Asking	Amazon, Google, Adobe, Microsoft
Difficulty	Easy-Medium
Time Complexity	$O(n)$
Space Complexity	$O(1)$ extra (in-place) or $O(n)$ for output
Problems Covered	String Compression (in-place), Run Length Encoding, Count and Say

INTUITION

- Compression problems ask you to represent repeated runs of characters compactly (e.g., "aaabbc" -> "a3b2c1") — the core skill is a single pass tracking run-start and run-length, writing output either in-place or to a new buffer.
- In-place variants (like LeetCode's String Compression) test careful index management: a read pointer scanning ahead and a write pointer building the compacted result behind it, never overtaking.

PATTERN RECOGNITION — Trigger Words

- compress repeated characters
- run length encoding
- in-place with $O(1)$ extra space

If interviewer says "compress the string in-place, return the new length" — think two-pointer (read/write) run-length tracking, not building an auxiliary string.

DRY RUN

```
chars=['a','a','b','b','c','c','c']
run 'a' x2 -> write 'a','2'
run 'b' x2 -> write 'b','2'
run 'c' x3 -> write 'c','3'
final length=6
```

BRUTE FORCE

- Build a brand-new string with string concatenation ($+=$) inside a loop — each $+=$ can be $O(n)$, leading to $O(n^2)$ in the worst case for naive implementations.

Time: $O(n^2)$ naive concatenation | Space: $O(n)$

BETTER SOLUTION

- Use a stringstream or reserve capacity upfront to avoid repeated reallocation — correct $O(n)$ but still uses auxiliary space.

Time: $O(n)$ | Space: $O(n)$

OPTIMAL SOLUTION

- In-place two-pointer: a slow 'write' pointer and a fast 'read' pointer that scans each run's length, writing the compressed run directly back into the same array.

Time: $O(n)$ | Space: $O(1)$ extra

C++ CODE (Optimal)

pattern19_optimal.cpp

```
int compress(vector<char>& chars) {
    int write = 0, read = 0, n = chars.size();
    while (read < n) {
        char c = chars[read];
        int count = 0;
        while (read < n && chars[read] == c) { read++; count++; }
        chars[write++] = c;
        if (count > 1) {
            for (char digit : to_string(count)) chars[write++] = digit;
        }
    }
    return write;
}
```

COMMON MISTAKES

- Writing the count even when it's 1 (spec usually says omit the digit for single-character runs).
- Writing multi-digit counts as a single character instead of iterating over `to_string(count)`'s digits.
- Using string concatenation (`+=`) in a loop causing hidden $O(n^2)$ behavior instead of in-place writes.
- Confusing the read and write pointers, overwriting characters before they've been read.
- Not handling the very last run after the loop ends (off-by-one, forgetting to flush the final run).
- Assuming input is only lowercase letters when digits/mixed characters might appear.
- Ignoring that 'in-place' means the RETURN VALUE (new length) matters, not just the compressed prefix of the array.
- Not clarifying whether compression should happen even if the result is LONGER than the original (spec-dependent).

INTERVIEW TIPS

- Clarify explicitly: does a single-character run still get a '1' suffix, or is it omitted? (Usually omitted — confirm with interviewer.)
- State the two-pointer invariant: 'write never overtakes read, so in-place writing is always safe.'
- Mention that naive `+=` string building can hide $O(n^2)$ behavior — shows awareness of amortized cost pitfalls.
- Walk through the multi-digit count case explicitly (e.g., a run of length 12) to show you handle it correctly.

FOLLOW-UP QUESTIONS

- How would you handle decompression (reverse operation) efficiently?
- What if the compressed result could be longer than the input — should you return the original instead?
- How would you extend this to streaming input where you don't know the whole string upfront?

VARIATIONS

- Easy: Count and Say (LeetCode 38)
- Medium: String Compression (LeetCode 443, in-place)
- Hard: Run Length Encoding with streaming/chunked input reconstruction

PATTERN SUMMARY

Compression is a single-pass run-length tracking problem — the in-place read/write two-pointer technique is the $O(1)$ -extra-space interview-grade answer.

PATTERN 20

Run Length Encoding (Advanced)

FAANG Frequency	Low-Medium
LeetCode Importance	Medium
Codeforces Importance	Medium
Companies Asking	Adobe, Amazon, Microsoft
Difficulty	Medium
Time Complexity	$O(n)$ encode/decode
Space Complexity	$O(n)$ for output
Problems Covered	Encode and Decode Strings, String Compression II (DP variant), RLE-based pattern queries

INTUITION

- Beyond basic RLE (Pattern 19), advanced RLE problems ask you to OPTIMIZE the encoding (e.g., minimum deletions to make the RLE as short as possible — String Compression II) which requires DP over run boundaries, not just a greedy single pass.
- RLE is also foundational for Encode/Decode Strings where you must make the encoding UNAMBIGUOUS (self-delimiting), a subtly different concern than compactness.

PATTERN RECOGNITION — Trigger Words

- minimum deletions for shortest encoding
- self-delimiting encoding for arbitrary strings
- encode a list of strings into one string safely

If interviewer says “encode a list of strings so you can always decode them back unambiguously, even with special characters” — think length-prefixed encoding, e.g. '5#hello3#cat'.

DRY RUN

```
encode(["hello", "cat"]) -> "5#hello3#cat"
decode: read digits until '#', that's the length, take exactly that many chars next, repeat.
```

BRUTE FORCE

- Use a delimiter character (like comma) to join strings — breaks if the delimiter itself can appear inside a string (ambiguous decode).

Time: $O(n)$ | Space: $O(n)$

BETTER SOLUTION

- Escape special characters manually — works but is fragile and easy to get wrong with nested escaping edge cases.

Time: $O(n)$ | Space: $O(n)$

OPTIMAL SOLUTION

- Length-prefixed encoding: store each string's length followed by a delimiter, then the raw string — fully unambiguous regardless of string content, no escaping needed.

Time: $O(n)$ total characters | Space: $O(n)$

C++ CODE (Optimal)

pattern20_optimal.cpp

```
string encode(vector<string>& strs) {
    string result;
    for (auto &s : strs) result += to_string(s.size()) + "#" + s;
    return result;
}
vector<string> decode(string s) {
    vector<string> result;
    int i = 0;
    while (i < (int)s.size()) {
        int j = i;
        while (s[j] != '#') j++;
        int len = stoi(s.substr(i, j - i));
        result.push_back(s.substr(j + 1, len));
        i = j + 1 + len;
    }
    return result;
}
```

COMMON MISTAKES

- Using a plain delimiter (comma, semicolon) without length-prefixing — breaks the moment a string contains that delimiter.
- Off-by-one when slicing the substring after the '#' delimiter (forgetting +1 for the delimiter itself).
- Not handling empty strings in the list (length 0 is still valid and must round-trip correctly).
- Assuming string length always fits cleanly without considering multi-digit lengths (10+, 100+ character strings).
- For String Compression II (DP variant), trying a greedy deletion strategy instead of proper DP over (index, deletions used) states.
- Forgetting that RLE encoding length calculation must account for multi-digit run lengths (a run of 10 contributes 2 digit characters, not 1).
- Not clarifying whether the encoding needs to be human-readable or just machine-decodable.
- Ignoring memory/performance cost of repeated string concatenation for very large lists (use a stringstream or reserve capacity).

INTERVIEW TIPS

- Explicitly state why comma/delimiter-only encoding is broken: 'if a string itself contains a comma, decode becomes ambiguous.'
- Walk through the length-prefix decode logic step by step on the whiteboard with a tricky example (a string containing digits or '#').
- For String Compression II, mention it requires DP since decisions about which characters to delete interact non-locally with run lengths.
- Mention real-world parallels: HTTP chunked transfer encoding uses this same length-prefix idea.

FOLLOW-UP QUESTIONS

- How would you handle strings containing arbitrary binary data (not just printable ASCII)?
- For String Compression II, how would you set up the DP state and transitions?
- How would this encoding scheme need to change for streaming (can't seek backward)?

VARIATIONS

- Easy: Basic Run Length Encoding of a single string
- Medium: Encode and Decode Strings (LeetCode 271)
- Hard: String Compression II (LeetCode 1531) — DP over deletions and run boundaries

PATTERN SUMMARY

Robust encoding must be self-delimiting, not just compact — length-prefixing is the unambiguous, interview-safe pattern; optimizing RLE length itself needs DP.

PATTERN 21

Pattern Matching (General)

FAANG Frequency	High
LeetCode Importance	High
Codeforces Importance	High
Companies Asking	Google, Amazon, Meta, Microsoft
Difficulty	Medium
Time Complexity	$O(n+m)$ to $O(n*m)$
Space Complexity	$O(m)$
Problems Covered	Implement strStr(), Shortest Distance to Character, Find the Index of the First Occurrence

INTUITION

- General pattern matching is the umbrella problem that KMP, Z-algorithm, Rabin-Karp, and even brute-force all solve differently — an interview usually starts here before narrowing to a specific algorithm based on your follow-up answers.
- Your job is to recognize which of the specialized algorithms (Patterns 13-16) applies based on the constraints given (single pattern vs multiple, guaranteed worst case vs average case, streaming vs static).

PATTERN RECOGNITION — Trigger Words

- find occurrence of pattern in text
- first/all matching positions
- substring search

If interviewer just says “find pattern P in text T ” with no further constraints — start with brute force, then proactively offer KMP/Rabin-Karp as the optimization.

DRY RUN

```
text="aaaaab", pattern="aaab"
naive matching fails repeatedly at position 4 in pattern, restarting from next text index each
time —  $O(n*m)$  worst case demo.
```

BRUTE FORCE

- For every starting index in text, compare characters one by one until mismatch or full match.

Time: $O(n*m)$ | Space: $O(1)$

BETTER SOLUTION

- Rabin-Karp for average-case $O(n+m)$ with hash pre-filtering.

Time: $O(n+m)$ average | Space: $O(1)$

OPTIMAL SOLUTION

- KMP for guaranteed $O(n+m)$ worst case via LPS array, or Z-algorithm via concatenation trick — pick based on what the interviewer emphasizes (worst-case guarantee vs elegance/CP-style).

Time: $O(n+m)$ | Space: $O(m)$

C++ CODE (Optimal)

`pattern21_optimal.cpp`

```
// Reference: naive baseline, escalate to KMP (Pattern 15) when pushed for guaranteed  $O(n+m)$ 
int naiveSearch(const string &text, const string &pat) {
    int n = text.size(), m = pat.size();
    for (int i = 0; i + m <= n; i++) {
        int j = 0;
        while (j < m && text[i + j] == pat[j]) j++;
        if (j == m) return i;
    }
    return -1;
}
```

COMMON MISTAKES

- Jumping straight to KMP without first establishing the brute-force baseline and its failure mode — interviewers want to see the escalation story.
- Not asking clarifying questions about pattern count, text size, or streaming constraints before choosing an algorithm.
- Mixing up when Rabin-Karp's average case is 'good enough' vs when KMP's guarantee is required.
- Forgetting edge cases: empty pattern (matches everywhere at index 0), pattern longer than text (no match possible).
- Not mentioning built-in options (`std::string::find`, which is typically well-optimized) when appropriate for a systems context.
- Assuming ASCII when Unicode pattern matching requires different byte-width handling.
- Ignoring case-insensitive matching requirements if not clarified upfront.
- Overcomplicating a simple single-search question with unnecessary suffix structures.

INTERVIEW TIPS

- Always start by stating the brute-force approach and WHY it can degrade to $O(n*m)$ — this sets up your optimization story naturally.
- Ask: 'will this pattern be searched once, or many times against different texts?' — the answer determines whether preprocessing pattern (KMP) or preprocessing text (suffix structures) makes sense.
- Explicitly compare KMP vs Rabin-Karp vs Z-algorithm trade-offs when asked which you'd pick and why.
- Mention `std::string::find` as the practical real-world answer, then explain what's under the hood conceptually.

FOLLOW-UP QUESTIONS

- Which algorithm would you pick if this pattern is searched against 1 million different texts?
- What if the pattern can contain wildcards?
- How would case-insensitivity change your preprocessing?

VARIATIONS

- Easy: Implement `strStr()` (LeetCode 28)
- Medium: Shortest Distance to a Character (position-relative pattern matching)
- Hard: Multi-pattern matching across a huge text stream (Aho-Corasick, see Pattern 24 backtracking notes)

PATTERN SUMMARY

General pattern matching is a decision problem: start with brute force, then justify escalating to KMP, Rabin-Karp, or Z-algorithm based on the actual constraints given.

PATTERN 22

Wildcard Matching

FAANG Frequency	High
LeetCode Importance	High
Codeforces Importance	Medium
Companies Asking	Google, Amazon, Meta, Microsoft
Difficulty	Hard
Time Complexity	$O(n*m)$
Space Complexity	$O(n*m)$ or $O(m)$ optimized
Problems Covered	Wildcard Matching (LeetCode 44), Glob-style pattern matching

INTUITION

- Wildcard matching ('?' matches any single character, '*' matches any sequence including empty) is fundamentally a 2D DP problem: $dp[i][j]$ represents whether $s[0..i)$ matches $p[0..j)$.
- The '*' transition is the crux: it can match EITHER zero characters (skip the star) OR one more character (extend the match) — $dp[i][j] = dp[i][j-1] \parallel dp[i-1][j]$.

PATTERN RECOGNITION — Trigger Words

- '?' matches any character
- '*' matches any sequence
- glob-style pattern
- wildcard matching

If interviewer says "implement wildcard pattern matching with ? and *" — think 2D DP table with the specific '*' dual-transition rule.

DRY RUN

```
s="aa", p="a*"
dp[0][0]=true (empty matches empty)
dp[1][1]=true ('a' matches 'a')
dp[1][2]=true ('*' can match zero chars, dp[1][1])
dp[2][2]=true ('*' extends to match second 'a', dp[1][2])
```

BRUTE FORCE

- Recursive backtracking trying both '*' interpretations (skip vs consume) without memoization — exponential blowup on adversarial patterns like '*a*a*a*a*'.

Time: $O(2^{(n+m)})$ worst case | Space: $O(n+m)$ recursion stack

BETTER SOLUTION

- Add memoization to the recursive solution (top-down DP) — correct and reduces to polynomial time.

Time: $O(n*m)$ | Space: $O(n*m)$

OPTIMAL SOLUTION

- Bottom-up 2D DP table, optionally compressed to $O(m)$ space using rolling rows since each row only depends on the previous row.

Time: $O(n*m)$ | Space: $O(m)$ with row compression

C++ CODE (Optimal)

pattern22_optimal.cpp

```
bool isMatch(string s, string p) {
    int n = s.size(), m = p.size();
    vector<vector<bool>> dp(n + 1, vector<bool>(m + 1, false));
    dp[0][0] = true;
    for (int j = 1; j <= m; j++)
        if (p[j - 1] == '*') dp[0][j] = dp[0][j - 1];
    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= m; j++) {
            if (p[j - 1] == '*') {
                dp[i][j] = dp[i][j - 1] || dp[i - 1][j];
            } else if (p[j - 1] == '?' || p[j - 1] == s[i - 1]) {
                dp[i][j] = dp[i - 1][j - 1];
            }
        }
    }
    return dp[n][m];
}
```

COMMON MISTAKES

- Forgetting the base case row: `dp[0][j]` must handle leading '*' characters in the pattern matching empty string.
- Getting the '*' transition backward — it's `dp[i][j-1]` (skip star) OR `dp[i-1][j]` (star consumes one more char), not AND.
- Using plain recursion without memoization, causing TLE on patterns with many consecutive '*' characters.
- Off-by-one between 0-indexed strings and 1-indexed DP table (classic string-DP bug).
- Not collapsing consecutive '*' characters in the pattern first (optional optimization, reduces redundant states).
- Confusing wildcard matching ('*' = any sequence) with regex matching ('*' = zero or more of PRECEDING character) — different semantics entirely (see Pattern 23).
- Forgetting `dp[0][0] = true` as the base case (empty matches empty).
- Not considering the greedy two-pointer alternative (track last star position) for $O(1)$ space, which some interviewers expect as the 'optimal' follow-up.

INTERVIEW TIPS

- Explicitly state the DP recurrence for '*' out loud before coding: 'star matches either zero characters or one more character, hence the OR of two subproblems.'
- Mention the $O(1)$ -space greedy alternative (tracking the last seen '*' position and backtrack point) as a follow-up optimization — shows depth beyond textbook DP.
- Clearly distinguish this from Regex Matching (Pattern 23) when asked — the semantics of '*' differ completely.
- Draw the DP table for a short tricky example (with consecutive stars) to build interviewer confidence.

FOLLOW-UP QUESTIONS

- Can you solve this with $O(1)$ extra space using a greedy two-pointer approach with backtracking on '*'?
- How does this differ from full regex matching with '.', '*' meaning 'zero or more of preceding'?
- How would you handle multiple wildcard patterns matched against the same string efficiently?

VARIATIONS

- Easy: Basic glob matching with only '?'
- Medium: Wildcard Matching (LeetCode 44) with both '?' and '*'
- Hard: Wildcard matching with additional character classes (like `[abc]`) requiring an extended DP/automaton

PATTERN SUMMARY

Wildcard matching is 2D DP where '*' branches into two subproblems (skip or consume) — know the $O(1)$ -space greedy alternative for the senior-level follow-up.

PATTERN 23

Regular Expression Matching

FAANG Frequency	High
LeetCode Importance	High
Codeforces Importance	Low-Medium
Companies Asking	Google, Meta, Amazon, Microsoft
Difficulty	Hard
Time Complexity	$O(n*m)$
Space Complexity	$O(n*m)$
Problems Covered	Regular Expression Matching (LeetCode 10)

INTUITION

- Regex matching with '.' (any single char) and '*' (zero or more of the PRECEDING character, not 'any sequence' like wildcard) requires a DP recurrence that looks one character back in the pattern whenever it encounters '*'.
- The '*' transition here is genuinely trickier than wildcard matching: $dp[i][j]$ when $p[j-1] == '*'$ depends on EITHER skipping the 'char*' pair entirely OR matching one instance of the preceding character and staying on the same pattern position.

PATTERN RECOGNITION — Trigger Words

- '.' matches any character
- '*' means zero or more of preceding element
- regex-style matching
- full match required (not partial)

If interviewer says "implement regex matching with . and * where * refers to the preceding character" — think 2D DP with a special two-case '*' transition that looks back one pattern character.

DRY RUN

```
s="aab", p="c*a*b"
'c*' can match zero 'c's (skip) -> effectively p becomes "a*b"
'a*' matches "aa" -> then 'b' matches 'b' -> true
```

BRUTE FORCE

- Recursive backtracking without memoization, branching on every '*' occurrence for zero-vs-more matches — exponential blowup.

Time: $O(2^{(n+m)})$ | Space: $O(n+m)$ recursion depth

BETTER SOLUTION

- Top-down memoized recursion — correct, avoids recomputation of overlapping subproblems.

Time: $O(n*m)$ | Space: $O(n*m)$

OPTIMAL SOLUTION

- Bottom-up 2D DP table built directly, with careful handling of the 'char*' pair as a UNIT during transitions.

Time: $O(n*m)$ | Space: $O(n*m)$

C++ CODE (Optimal)

`pattern23_optimal.cpp`

```
bool isMatch(string s, string p) {
    int n = s.size(), m = p.size();
    vector<vector<bool>> dp(n + 1, vector<bool>(m + 1, false));
    dp[0][0] = true;
    for (int j = 1; j <= m; j++)
        if (p[j - 1] == '*') dp[0][j] = dp[0][j - 2];
    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= m; j++) {
            if (p[j - 1] == '*') {
                bool zeroOccurrence = dp[i][j - 2];
                bool oneOrMore = (p[j - 2] == '.' || p[j - 2] == s[i - 1]) && dp[i - 1][j];
                dp[i][j] = zeroOccurrence || oneOrMore;
            } else if (p[j - 1] == '.' || p[j - 1] == s[i - 1]) {
                dp[i][j] = dp[i - 1][j - 1];
            }
        }
    }
    return dp[n][m];
}
```

COMMON MISTAKES

- Treating '*' the same as wildcard matching's '*' (any sequence) instead of 'zero or more of the PRECEDING character' — fundamentally different semantics.
- Accessing `p[j-2]` without checking `j >= 2` first, causing out-of-bounds when '*' appears as the first pattern character (invalid regex, but must guard defensively).
- Forgetting the base-case row must skip PAIRS (`j-2`) for '*' patterns matching empty string, not single positions.
- Not treating 'char*' as an atomic two-character unit during transitions — partial handling causes subtle bugs.
- Confusing this with Wildcard Matching (Pattern 22) and copy-pasting the wrong '*' transition.
- Off-by-one between 0-indexed input strings and 1-indexed DP table.
- Not handling patterns that are technically malformed per problem constraints (though LeetCode guarantees valid patterns, real-world regex parsing must validate).
- Missing memoization in a recursive approach, leading to TLE on long alternating patterns.

INTERVIEW TIPS

- Explicitly state upfront: 'unlike wildcard matching, this * refers to the PRECEDING character, so I need to look back one position in the pattern.'
- Walk through the base-case row carefully on the whiteboard — it's the most bug-prone part (skipping in pairs for leading '*' patterns).
- Mention that full regex engines generalize this into an NFA/DFA construction (Thompson's construction) for production-grade implementations.
- Compare directly against Wildcard Matching when asked — shows you don't conflate similar-looking problems.

FOLLOW-UP QUESTIONS

- How would production regex engines (like RE2) implement this more efficiently using automata instead of DP?
- How would you extend this to support '+' and '{n,m}' repetition operators?
- What's the difference in DP transition between this and Wildcard Matching, precisely?

VARIATIONS

- Easy: Matching with only '.' (no '*')
- Medium: Regular Expression Matching (LeetCode 10) full problem
- Hard: Extended regex with '+', character classes, and anchors requiring NFA/DFA construction

PATTERN SUMMARY

Regex '*' means zero-or-more of the PRECEDING character — the DP must treat 'char*' as an atomic pair, the single most important distinction from Wildcard Matching.

PATTERN 24

Backtracking on Strings

FAANG Frequency	High
LeetCode Importance	High
Codeforces Importance	Medium
Companies Asking	Google, Amazon, Meta, Microsoft
Difficulty	Medium-Hard
Time Complexity	Exponential worst case, pruned in practice
Space Complexity	$O(n)$ recursion depth
Problems Covered	Palindrome Partitioning, Letter Combinations of a Phone Number, Generate Parentheses, Word Search

INTUITION

- Backtracking explores the full decision tree of choices (which character/partition/path to take next) but PRUNES branches early the moment a partial solution becomes invalid — this is what makes it tractable despite exponential worst-case complexity.
- For strings specifically: partition problems branch on 'where to cut next', combination problems branch on 'which character to pick next', and grid problems (Word Search) branch on 'which direction to move next'.

PATTERN RECOGNITION — Trigger Words

- all possible partitions
- all combinations of characters
- generate all valid strings satisfying constraint X
- search a grid for a word

If interviewer says “partition the string such that every substring is a palindrome, return all such partitions” — think backtracking: at each position, try every valid palindromic prefix, recurse on the rest.

DRY RUN

```
s="aab"
try "a" (palindrome) -> recurse on "ab"
  try "a" -> recurse on "b" -> "b" valid -> ["a","a","b"]
  try "ab" -> not palindrome, skip
try "aa" (palindrome) -> recurse on "b" -> ["aa","b"]
```

BRUTE FORCE

- Generate every possible partition/combination first, THEN filter for validity — wastes work building invalid branches fully before checking.

Time: $O(2^n)$ or worse, unpruned | Space: $O(2^n)$ storing all partitions

BETTER SOLUTION

- Backtrack but recompute validity checks (e.g., palindrome check) from scratch at every recursive call — correct but redundant work.

Time: $O(2^n * n)$ with repeated palindrome checks | Space: $O(n)$ recursion depth

OPTIMAL SOLUTION

- Backtrack WITH precomputed validity (e.g., a palindrome DP table built once in $O(n^2)$) so each recursive branch check is $O(1)$, and prune the moment a partial choice is invalid.

Time: $O(n^2)$ precompute + $O(2^n)$ worst-case exploration, heavily pruned in practice | Space: $O(n^2)$ for the precomputed table + $O(n)$ recursion depth

C++ CODE (Optimal)

pattern24_optimal.cpp

```
void backtrack(int start, string &s, vector<string> &path,
               vector<vector<string>> &result, vector<vector<bool>> &isPal) {
    if (start == (int)s.size()) { result.push_back(path); return; }
    for (int end = start; end < (int)s.size(); end++) {
        if (isPal[start][end]) {
            path.push_back(s.substr(start, end - start + 1));
            backtrack(end + 1, s, path, result, isPal);
            path.pop_back(); // undo choice
        }
    }
}

vector<vector<string>> partition(string s) {
    int n = s.size();
    vector<vector<bool>> isPal(n, vector<bool>(n, false));
    for (int i = n - 1; i >= 0; i--)
        for (int j = i; j < n; j++)
            isPal[i][j] = (s[i] == s[j]) && (j - i < 2 || isPal[i + 1][j - 1]);
    vector<vector<string>> result;
    vector<string> path;
    backtrack(0, s, path, result, isPal);
    return result;
}
```

COMMON MISTAKES

- Forgetting to precompute palindrome validity, redoing an $O(n)$ check inside every recursive call (adds an unnecessary factor to already-exponential complexity).
- Missing the `path.pop_back()` undo step after recursing — the single most common backtracking bug, corrupts all sibling branches.
- Off-by-one in substring bounds when building `s.substr(start, end - start + 1)`.
- Not pruning early — checking validity only after fully building a candidate instead of checking at each step.
- Confusing 'return all valid partitions' (needs full exploration) with 'is there at least one valid partition' (can early-exit on first success).
- Using global mutable state carelessly without properly resetting it between independent recursive branches.
- Not considering memoization when subproblems repeat significantly (though pure enumeration problems often can't be memoized the same way as optimization problems).
- Ignoring stack depth concerns for very long strings (recursion depth = string length in the worst case).

INTERVIEW TIPS

- Always state the three backtracking pillars out loud: 'choose, explore, un-choose' — interviewers listen for this exact mental model.
- Mention precomputing palindrome validity via DP BEFORE backtracking — shows you optimize the pruning condition itself, not just the traversal.
- For grid-based word search, mention marking visited cells and UNMARKING them on backtrack (same undo principle, applied to a grid instead of a string).
- Distinguish clearly between 'find any valid answer' (early exit) and 'find all valid answers' (full exploration) — ask which is needed.

FOLLOW-UP QUESTIONS

- How would you modify this to return just the MINIMUM number of cuts instead of all partitions? (Different DP, see Pattern 25/26 area)
- How would Word Search's backtracking change if diagonal moves were also allowed?
- Can you bound the worst-case number of partitions for a string of all-identical characters?

VARIATIONS

- Easy: Generate Parentheses (LeetCode 22)
- Medium: Palindrome Partitioning (LeetCode 131)
- Hard: Word Search II combining Trie pruning with grid backtracking (LeetCode 212)

PATTERN SUMMARY

Backtracking on strings is choose-explore-un-choose over partitions, combinations, or paths — precomputing validity checks is what separates a correct solution from a FAST one.

PATTERN 25

DP on Strings (Foundations)

FAANG Frequency	Very High
LeetCode Importance	Very High
Codeforces Importance	High
Companies Asking	Google, Amazon, Meta, Microsoft, Adobe
Difficulty	Medium-Hard
Time Complexity	$O(n*m)$ typical
Space Complexity	$O(n*m)$, often optimizable to $O(\min(n,m))$
Problems Covered	Longest Common Subsequence, Edit Distance, Distinct Subsequences, Interleaving String

INTUITION

- String DP problems almost always reduce to a 2D table $dp[i][j]$ representing 'the answer using the first i characters of string A and first j characters of string B' — the transition compares $s1[i-1]$ vs $s2[j-1]$ and combines smaller subproblems.
- The recurring skeleton is: if characters match, extend a diagonal subproblem; if they don't, combine adjacent subproblems (left, up, or diagonal) depending on whether insert/delete/replace operations are allowed.

PATTERN RECOGNITION — Trigger Words

- two strings, comparing/transforming one into another
- longest common ...
- number of ways to form/interleave
- minimum operations to transform

If interviewer says "compare two strings and compute some optimal value (length, count, cost)" — think 2D DP table indexed by prefix lengths of both strings.

DRY RUN

```
s1="abc", s2="ac"
dp[i][j] = LCS length of s1[0..i) and s2[0..j)
dp[1][1]: 'a'=='a' -> dp[1][1]=dp[0][0]+1=1
dp[3][2]: 'c'=='c' -> dp[3][2]=dp[2][1]+1=2
```

BRUTE FORCE

- Recursively try every combination of including/excluding characters from both strings without memoization — exponential.

Time: $O(2^{(n+m)})$ | Space: $O(n+m)$ recursion depth

BETTER SOLUTION

- Top-down memoized recursion over (i,j) states — correct, avoids recomputation.

Time: $O(n*m)$ | Space: $O(n*m)$

OPTIMAL SOLUTION

- Bottom-up 2D DP table; further optimized to two rolling rows ($O(\min(n,m))$ space) since each row/state depends only on the previous row.

Time: $O(n*m)$ | Space: $O(\min(n,m))$ optimized

C++ CODE (Optimal)

pattern25_optimal.cpp

```
int longestCommonSubsequence(string s1, string s2) {
    int n = s1.size(), m = s2.size();
    vector<vector<int>> dp(n + 1, vector<int>(m + 1, 0));
    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= m; j++) {
            if (s1[i - 1] == s2[j - 1]) dp[i][j] = dp[i - 1][j - 1] + 1;
            else dp[i][j] = max(dp[i - 1][j], dp[i][j - 1]);
        }
    }
    return dp[n][m];
}
```

COMMON MISTAKES

- Trying to solve string DP problems with greedy reasoning first — most don't have the greedy-choice property (subtle counterexamples exist).
- Off-by-one between 0-indexed strings and 1-indexed DP table (the most common string-DP bug across ALL these problems).
- Forgetting the base case row/column (empty string comparisons) which anchors the entire table.
- Not recognizing which of the 3 neighbors (diagonal, up, left) apply for a given problem variant (LCS uses $\text{diagonal} + \max(\text{up}, \text{left})$; Edit Distance uses all three with costs).
- Space-optimizing to rolling rows prematurely, before getting the 2D version correct and verified.
- Confusing 'subsequence' (skip allowed, order preserved) with 'substring' (contiguous) — completely different DP formulations.
- Not tracing back through the DP table when the ACTUAL sequence/string (not just length/count) is required.
- Assuming all string-DP problems share the exact same transition — each problem's transition must be derived from its specific rules.

INTERVIEW TIPS

- Always define the DP state in words FIRST out loud: 'dp[i][j] represents...' — interviewers grade this articulation heavily.
- Derive the transition by reasoning about the LAST character decision (match, skip s1, skip s2) rather than memorizing formulas.
- Mention the space optimization (rolling array) as a follow-up, but get the correct 2D version working first.
- If the actual sequence (not just its length) is needed, mention backtracking through the filled DP table.

FOLLOW-UP QUESTIONS

- How would you reconstruct the actual LCS string, not just its length, from this DP table?
- How would you optimize this to $O(\min(n, m))$ space?
- How does this transition change for Edit Distance where insert/delete/replace all have different costs?

VARIATIONS

- Easy: Longest Common Subsequence (LeetCode 1143)
- Medium: Distinct Subsequences (LeetCode 115)
- Hard: Interleaving String (LeetCode 97) — 3-way DP state consideration

PATTERN SUMMARY

String DP = define $\text{dp}[i][j]$ over prefix lengths of both strings, derive transitions from the last-character decision — the master template behind LCS, Edit Distance, and dozens more.

PATTERN 26

Longest Common Subsequence (LCS)

FAANG Frequency	Very High
LeetCode Importance	Very High
Codeforces Importance	High
Companies Asking	Google, Amazon, Meta, Microsoft
Difficulty	Medium
Time Complexity	$O(n*m)$
Space Complexity	$O(n*m)$, optimizable to $O(\min(n,m))$
Problems Covered	Longest Common Subsequence, Shortest Common Supersequence, Delete Operation for Two Strings

INTUITION

- LCS asks for the longest sequence of characters (not necessarily contiguous, but order-preserved) common to both strings — it's the canonical example of the string-DP template from Pattern 25.
- LCS is also the SECRET FOUNDATION behind many other problems: Edit Distance, Shortest Common Supersequence, and Diff/version-control algorithms (like git diff) all build on LCS reasoning.

PATTERN RECOGNITION — Trigger Words

- longest common subsequence
- characters in the same relative order (not contiguous)
- diff between two texts/versions

If interviewer says "find the longest subsequence common to both strings" — think 2D DP: match extends diagonal, mismatch takes max of skipping from either string.

DRY RUN

```
s1="abcde", s2="ace"
dp table fills to 3 ("ace" is a subsequence of both)
trace back through matches to reconstruct: e<-c<-a
```

BRUTE FORCE

- Try every subsequence of s1 (2^n of them) and check if it's also a subsequence of s2 — exponential.

Time: $O(2^n * m)$ | Space: $O(n)$

BETTER SOLUTION

- Top-down memoized recursion on (i,j) prefix indices.

Time: $O(n*m)$ | Space: $O(n*m)$

OPTIMAL SOLUTION

- Bottom-up 2D DP table; space-optimized to two rolling rows since each row depends only on the row above.

Time: $O(n*m)$ | Space: $O(\min(n,m))$

C++ CODE (Optimal)

pattern26_optimal.cpp

```
int lcsLength(string s1, string s2) {
    int n = s1.size(), m = s2.size();
    vector<int> prev(m + 1, 0), curr(m + 1, 0);
    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= m; j++) {
            if (s1[i - 1] == s2[j - 1]) curr[j] = prev[j - 1] + 1;
            else curr[j] = max(prev[j], curr[j - 1]);
        }
        prev = curr;
    }
    return prev[m];
}
```

COMMON MISTAKES

- Confusing LCS (subsequence, skip allowed) with Longest Common Substring (contiguous, Pattern 27) — completely different DP transitions.
- Forgetting to reset/copy the curr row properly when space-optimizing to rolling rows.
- Off-by-one between string indices and DP table indices.
- Trying a greedy 'match earliest occurrence' approach, which fails on many valid counterexamples.
- Not handling the reconstruction (actual subsequence string) correctly — backtrack from $dp[n][m]$, moving diagonally on match, otherwise toward the larger neighbor.
- Using $O(n*m)$ space by default when the interviewer explicitly asks for optimized space.
- Assuming LCS length directly gives edit distance — related but requires separate reasoning (edit distance = $n + m - 2 * \text{LCS}$ only for insert/delete-only edits, not replace).
- Not recognizing when a problem is 'secretly' an LCS problem in disguise (e.g., Shortest Common Supersequence, Delete Operation for Two Strings).

INTERVIEW TIPS

- Explicitly distinguish 'subsequence' from 'substring' out loud before writing any code — this single sentence prevents a huge class of confusion.
- Mention that Edit Distance, Shortest Common Supersequence, and diff tools all build on this exact same DP skeleton — shows systems-level pattern recognition.
- If asked to reconstruct the actual LCS, explain the backtracking-through-the-table procedure clearly.
- Proactively offer the $O(\min(n,m))$ space optimization once the 2D version is verified correct.

FOLLOW-UP QUESTIONS

- How would you reconstruct the actual LCS string (not just its length)?
- How does LCS relate to Edit Distance mathematically (when only insertions/deletions are allowed)?
- How would you find the LCS of three strings instead of two?

VARIATIONS

- Easy: Longest Common Subsequence (LeetCode 1143)
- Medium: Delete Operation for Two Strings (LeetCode 583)
- Hard: Shortest Common Supersequence (LeetCode 1092) reconstruction via LCS backtracking

PATTERN SUMMARY

LCS is the parent pattern behind Edit Distance and diff algorithms — master the match-extends-diagonal, mismatch-takes-max transition and its backtracking reconstruction.

PATTERN 27

Longest Common Substring

FAANG Frequency	Medium
LeetCode Importance	Medium
Codeforces Importance	High
Companies Asking	Google, Amazon, Adobe
Difficulty	Medium
Time Complexity	$O(n \cdot m)$ DP or $O(n+m)$ with suffix structures
Space Complexity	$O(n \cdot m)$ or $O(\min(n,m))$ optimized
Problems Covered	Longest Common Substring (GeeksforGeeks classic), Maximum Length of Repeated Subarray

INTUITION

- Unlike LCS (Pattern 26), a common SUBSTRING must be CONTIGUOUS in both strings — this changes the DP transition fundamentally: on a mismatch, the running match must reset to ZERO rather than carrying forward the max of neighbors.
- The answer isn't $dp[n][m]$ (like LCS) but the MAXIMUM value seen anywhere in the entire DP table, since the longest common substring could end at any position.

PATTERN RECOGNITION — Trigger Words

- longest common substring (contiguous, not subsequence)
- maximum length of repeated subarray
- longest matching contiguous run between two sequences

If interviewer emphasizes 'CONTIGUOUS' explicitly — immediately flag that this is NOT the LCS transition; mismatches must reset the running length to zero.

DRY RUN

```
s1="abcdxyz", s2="xyzabcd"
dp[i][j] tracks length of common substring ENDING at s1[i-1], s2[j-1]
matches build a diagonal run of length 4 ("abcd"), and separately length 3 ("xyz")
max over the whole table = 4
```

BRUTE FORCE

- Check every pair of substrings from both strings for equality directly — $O(n^2 * m^2)$ naive, or $O(n^2 * m)$ with smarter substring generation.

Time: $O(n^2 * m^2)$ | Space: $O(1)$

BETTER SOLUTION

- Use rolling hash (Pattern 13) to compare substrings of a fixed length in $O(1)$ after hashing, combined with binary search on the answer length.

Time: $O((n+m) \log(\min(n,m)))$ | Space: $O(n+m)$

OPTIMAL SOLUTION

- 2D DP where $dp[i][j] = dp[i-1][j-1] + 1$ on match, else 0 (hard reset) — track the running maximum across the whole table as you go.

Time: $O(n*m)$ | Space: $O(\min(n,m))$ with rolling rows

C++ CODE (Optimal)

`pattern27_optimal.cpp`

```
int longestCommonSubstring(string s1, string s2) {
    int n = s1.size(), m = s2.size(), best = 0;
    vector<int> prev(m + 1, 0), curr(m + 1, 0);
    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= m; j++) {
            if (s1[i - 1] == s2[j - 1]) {
                curr[j] = prev[j - 1] + 1;
                best = max(best, curr[j]);
            } else {
                curr[j] = 0; // hard reset on mismatch -- the key difference from LCS
            }
        }
        prev = curr;
    }
    return best;
}
```

COMMON MISTAKES

- Copy-pasting the LCS transition ($\max(\text{up}, \text{left})$ on mismatch) instead of the hard-reset-to-zero rule required for CONTIGUOUS substrings.
- Returning $\text{dp}[n][m]$ instead of the MAXIMUM value seen anywhere in the table — the longest common substring can end at any (i, j) , not necessarily the last.
- Forgetting to reset $\text{curr}[j]$ to 0 explicitly on mismatch when reusing a rolling array (stale values from the previous row leak through).
- Not considering the rolling-hash + binary-search alternative when asked for a faster or more scalable approach.
- Off-by-one between string indices and DP table indices.
- Confusing this problem with Longest Common Subsequence in conversation, using the wrong name when explaining the approach.
- Ignoring memory costs of full $O(n*m)$ table for very large strings without mentioning the rolling-row optimization.
- Not tracking WHERE the maximum occurred if the actual substring (not just length) needs to be returned.

INTERVIEW TIPS

- Say explicitly and early: 'since this needs to be CONTIGUOUS, mismatches reset the count to zero — this is different from LCS.'
- Track the best length AND its ending position during the DP fill if the actual substring is required as output.
- Mention the rolling-hash + binary-search-on-length alternative as a scalable alternative for very long strings.
- Clarify with the interviewer immediately whether 'common substring' or 'common subsequence' is meant — the terms are often confused even by interviewers themselves.

FOLLOW-UP QUESTIONS

- How would you extract the actual longest common substring (not just its length) from the DP table?
- How would you solve this using a Generalized Suffix Array instead of DP?
- How would this generalize to finding the longest common substring among THREE OR MORE strings?

VARIATIONS

- Easy: Maximum Length of Repeated Subarray (LeetCode 718, array version of same idea)
- Medium: Longest Common Substring (classic GfG/interview problem)
- Hard: Longest Common Substring among k strings using Generalized Suffix Array + LCP

PATTERN SUMMARY

Longest Common SUBSTRING resets to zero on mismatch and tracks a table-wide maximum — the single transition change that distinguishes it from LCS.

PATTERN 28

Edit Distance

FAANG Frequency	Very High
LeetCode Importance	Very High
Codeforces Importance	High
Companies Asking	Google, Amazon, Meta, Microsoft, Adobe
Difficulty	Hard
Time Complexity	$O(n*m)$
Space Complexity	$O(n*m)$, optimizable to $O(\min(n,m))$
Problems Covered	Edit Distance (LeetCode 72), One Edit Distance, Delete Operation for Two Strings

INTUITION

- Edit Distance asks the minimum number of insert/delete/replace operations to transform one string into another — the DP transition considers all THREE operations at each mismatched position and takes the minimum.
- This is LCS's more general sibling: where LCS only 'skips' characters implicitly, Edit Distance explicitly PAYS a cost for every insert, delete, or replace, giving a richer transition with three neighbor options instead of two.

PATTERN RECOGNITION — Trigger Words

- minimum operations to convert string A to string B
- insert delete replace
- spell-check / diff-style transformation cost

If interviewer says "what's the minimum number of edits to turn word1 into word2" — think 2D DP with three transition options (insert, delete, replace) at every mismatch.

DRY RUN

```
s1="horse", s2="ros"
dp[i][j] = min ops to convert s1[0..i) to s2[0..j)
on mismatch: dp[i][j] = 1 + min(dp[i-1][j] /*delete*/, dp[i][j-1] /*insert*/, dp[i-1][j-1] /*r
eplace*/)
final answer dp[5][3] = 3
```

BRUTE FORCE

- Recursive exploration of all three operations at every position without memoization — exponential blowup.

Time: $O(3^{(n+m)})$ | Space: $O(n+m)$ recursion depth

BETTER SOLUTION

- Top-down memoized recursion over (i,j) states — correct, eliminates repeated subproblem computation.

Time: $O(n*m)$ | Space: $O(n*m)$

OPTIMAL SOLUTION

- Bottom-up 2D DP table; space-optimizable to two rolling rows since each row depends only on the row above and the current row's previous column.

Time: $O(n*m)$ | Space: $O(\min(n,m))$

C++ CODE (Optimal)

pattern28_optimal.cpp

```
int minDistance(string word1, string word2) {
    int n = word1.size(), m = word2.size();
    vector<vector<int>> dp(n + 1, vector<int>(m + 1, 0));
    for (int i = 0; i <= n; i++) dp[i][0] = i; // delete all
    for (int j = 0; j <= m; j++) dp[0][j] = j; // insert all
    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= m; j++) {
            if (word1[i - 1] == word2[j - 1]) dp[i][j] = dp[i - 1][j - 1];
            else dp[i][j] = 1 + min({dp[i - 1][j], dp[i][j - 1], dp[i - 1][j - 1]});
        }
    }
    return dp[n][m];
}
```

COMMON MISTAKES

- Forgetting the base-case row/column ($dp[i][0]=i$, $dp[0][j]=j$) which represents converting to/from an empty string via pure inserts/deletes.
- Mixing up which neighbor corresponds to which operation ($dp[i-1][j]=\text{delete from } s1$, $dp[i][j-1]=\text{insert into } s1$, $dp[i-1][j-1]=\text{replace}$) — get this backward and the logic still 'runs' but reasoning about it breaks down.
- Not adding the +1 cost when characters mismatch (forgetting the operation has a cost at all).
- Confusing this with LCS's transition (max instead of $1+\min$) — a completely different optimization goal (minimize cost vs maximize length).
- Off-by-one between string indices and DP table indices.
- Assuming all three operations always cost 1 — some problem variants have DIFFERENT costs per operation type, requiring a weighted version of the same DP.
- Not considering the space optimization to rolling rows when explicitly asked for optimized space.
- Forgetting real-world context: this is literally the Levenshtein distance algorithm used in spell-checkers and diff tools — worth mentioning for systems-thinking credit.

INTERVIEW TIPS

- State the three operations and their corresponding DP neighbor explicitly: 'delete looks up, insert looks left, replace looks diagonal.'
- Mention this is the Levenshtein distance algorithm by name — used in real spell-checkers, DNA sequence alignment, and version control diffing.
- If costs differ per operation type, explain how the recurrence generalizes (multiply each term by its respective cost instead of assuming 1).
- Proactively offer the $O(\min(n,m))$ space optimization after the 2D table version is verified.

FOLLOW-UP QUESTIONS

- How would this change if insert/delete/replace had different costs?
- How would you reconstruct the actual sequence of edit operations, not just the count?
- How is this related to LCS mathematically, and when would edit distance equal $n + m - 2 * \text{LCS}$?

VARIATIONS

- Easy: One Edit Distance (check if strings are within 1 edit)
- Medium: Delete Operation for Two Strings (LeetCode 583, delete-only variant)
- Hard: Edit Distance with weighted/custom costs per operation type

PATTERN SUMMARY

Edit Distance generalizes LCS with an explicit cost per insert/delete/replace — the three-way min transition is the single most tested 'hard' string-DP problem in FAANG interviews.

PATTERN 29

Word Break

FAANG Frequency	Very High
LeetCode Importance	High
Codeforces Importance	Medium
Companies Asking	Google, Amazon, Meta, Microsoft
Difficulty	Medium-Hard
Time Complexity	$O(n^2)$ to $O(n^2 * k)$
Space Complexity	$O(n)$
Problems Covered	Word Break (LeetCode 139), Word Break II (return all sentences), Concatenated Words

INTUITION

- Word Break asks if a string can be segmented into a sequence of dictionary words — $dp[i] = \text{true}$ if $s[0..i)$ can be fully segmented, checked by trying every possible LAST word ending at position i .
- This is a 1D DP (unlike the 2D DP of LCS/Edit Distance) because there's only ONE string involved; the 'second dimension' is implicitly the dictionary lookup, best done via a hashset or Trie for $O(1)/O(k)$ word checks.

PATTERN RECOGNITION — Trigger Words

- can this string be segmented into dictionary words
- break string into valid words
- all ways to segment into words

If interviewer says “can 'leetcode' be segmented into words from a given dictionary” — think $dp[i] = \text{true}$ if some $dp[j]$ is true AND $s[j..i)$ is a dictionary word.

DRY RUN

```
s="leetcode", dict={"leet","code"}
dp[0]=true (empty prefix)
dp[4]=true ("leet" is in dict, dp[0] true)
dp[8]=true ("code" is in dict, dp[4] true) -> answer true
```

BRUTE FORCE

- Recursively try every possible split point without memoization, re-solving identical subproblems repeatedly — exponential.

Time: $O(2^n)$ | Space: $O(n)$ recursion depth

BETTER SOLUTION

- Top-down memoized recursion using a hashset for $O(1)$ dictionary lookups per candidate word.

Time: $O(n^2)$ average (n split points $\times$ n substring extraction cost) | Space: $O(n)$

OPTIMAL SOLUTION

- Bottom-up 1D DP table $dp[i]$, iterating all valid $j < i$ for each i , using a hashset (or Trie for large dictionaries) for word lookups.
- For Word Break II (return ALL sentences), add memoized backtracking on top of the same $dp[i]$ feasibility check to prune impossible branches early.

Time: $O(n^2)$ with hashset lookups, $O(n^2)$ substring cost dominates | Space: $O(n)$

C++ CODE (Optimal)

`pattern29_optimal.cpp`

```
bool wordBreak(string s, vector<string>& wordDict) {
    unordered_set<string> dict(wordDict.begin(), wordDict.end());
    int n = s.size();
    vector<bool> dp(n + 1, false);
    dp[0] = true;
    for (int i = 1; i <= n; i++) {
        for (int j = 0; j < i; j++) {
            if (dp[j] && dict.count(s.substr(j, i - j))) {
                dp[i] = true;
                break; // no need to check further j once true
            }
        }
    }
    return dp[n];
}
```

COMMON MISTAKES

- Forgetting `dp[0] = true` as the base case (empty prefix is trivially 'segmentable').
- Not breaking early once `dp[i]` is found true, redundantly checking remaining `j` values.
- Using substring extraction `s.substr(j, i-j)` repeatedly without considering the $O(n)$ cost per call adding up (fine for moderate `n`, but worth mentioning for very long strings).
- For Word Break II, forgetting to prune branches using the `dp[]` feasibility array FIRST — without this, backtracking alone can explode exponentially even when a valid segmentation exists.
- Not using a Trie when the dictionary is very large, missing the opportunity for prefix-based early termination during the inner loop.
- Confusing Word Break (existence check, boolean) with Word Break II (enumerate all valid sentences) — very different complexity profiles.
- Ignoring the classic exponential blowup case for Word Break II: a string like 'aaaaaaaaaaaaaaaaaab' with dictionary {'a','aa','aaa',...} without feasibility pruning.
- Not considering memoization on substrings for Word Break II to avoid recomputing the same suffix's valid segmentations repeatedly.

INTERVIEW TIPS

- State clearly: 'dp[i] means the prefix of length i can be fully segmented' — precise DP-state articulation matters here.
- For Word Break II, mention explicitly: 'I'll first compute the `dp[]` feasibility array, THEN backtrack only through feasible positions to avoid exponential blowup on infeasible strings.'
- Suggest a Trie-based dictionary for large word lists to enable early-exit during the substring scan.
- Mention real-world parallel: this is the core problem behind CJK (Chinese/Japanese) text segmentation where there are no spaces between words.

FOLLOW-UP QUESTIONS

- How would you return ALL valid segmentations (Word Break II) efficiently without exponential blowup?
- How would a Trie-based dictionary improve the inner loop's performance over a hashset?
- How would you solve this for a stream of characters where you can't look ahead arbitrarily?

VARIATIONS

- Easy: Basic Word Break existence check (LeetCode 139)
- Medium: Concatenated Words (LeetCode 472, word break applied per-word across a list)
- Hard: Word Break II (LeetCode 140) — return all sentences with feasibility-pruned backtracking

PATTERN SUMMARY

Word Break is 1D DP over segmentation feasibility — the critical skill for Word Break II is pruning backtracking using the feasibility array first, avoiding exponential blowup.

PATTERN 30

Minimum Window Substring

FAANG Frequency	Very High
LeetCode Importance	Very High
Codeforces Importance	Medium
Companies Asking	Google, Amazon, Meta, Microsoft, Uber
Difficulty	Hard
Time Complexity	$O(n+m)$
Space Complexity	$O(k)$ for character counts
Problems Covered	Minimum Window Substring (LeetCode 76), Smallest Window containing all characters of another string

INTUITION

- This is the hardest, most tested variant of Sliding Window (Pattern 05): find the SMALLEST window in *s* that contains ALL characters of *t* (with correct multiplicities) — requiring a 'need' frequency map and a 'have' counter tracking how many required character TYPES are currently satisfied.
- The key insight that separates a correct $O(n)$ solution from a slow one: track a single integer 'formed' counter (how many distinct required characters currently meet their needed count) instead of comparing entire frequency maps on every window change.

PATTERN RECOGNITION — Trigger Words

- smallest/minimum window containing all characters
- substring containing every character of another string
- shortest substring satisfying a coverage constraint

If interviewer says “find the smallest substring of *s* containing all characters of *t*” — think variable sliding window with a need-map and a formed-count integer, not repeated full-map comparisons.

DRY RUN

```
s="ADOBECODEBANC", t="ABC"
need={A:1,B:1,C:1}
expand right until formed==3 (all needs met), then shrink left while still valid, tracking the
smallest valid window seen
answer: "BANC"
```

BRUTE FORCE

- For every possible (left,right) pair, check if the window contains all needed characters — recomputing coverage from scratch each time.

Time: $O(n^2 * k)$ | Space: $O(k)$

BETTER SOLUTION

- Sliding window with a full frequency map comparison at each step to check if all needs are satisfied — correct but each check costs $O(k)$ (alphabet size), not $O(1)$.

Time: $O(n * k)$ | Space: $O(k)$

OPTIMAL SOLUTION

- Sliding window with a single 'formed' integer counter that increments/decrements in $O(1)$ exactly when a character's count crosses its required threshold — avoids ever comparing full maps.

Time: $O(n+m)$ | Space: $O(k)$

C++ CODE (Optimal)

pattern30_optimal.cpp

```
string minWindow(string s, string t) {
    unordered_map<char,int> need, window;
    for (char c : t) need[c]++;
    int required = need.size(), formed = 0;
    int left = 0, bestLen = INT_MAX, bestStart = 0;
    for (int right = 0; right < (int)s.size(); right++) {
        char c = s[right];
        window[c]++;
        if (need.count(c) && window[c] == need[c]) formed++;
        while (formed == required) {
            if (right - left + 1 < bestLen) { bestLen = right - left + 1; bestStart = left; }
            char lc = s[left];
            window[lc]--;
            if (need.count(lc) && window[lc] < need[lc]) formed--;
            left++;
        }
    }
    return bestLen == INT_MAX ? "" : s.substr(bestStart, bestLen);
}
```

COMMON MISTAKES

- Comparing the full 'window' and 'need' maps on every step instead of maintaining the $O(1)$ 'formed' counter — turns $O(n)$ into $O(n*k)$.
- Incrementing/decrementing 'formed' incorrectly — it should change ONLY when a character's count crosses its EXACT required threshold, not on every occurrence.
- Forgetting that 'window' can and should track characters NOT in 'need' too (harmless, but the formed-check must only look at `need.count(c)`).
- Off-by-one in window length calculation (`right - left + 1`).
- Not shrinking the window maximally once valid — must shrink in a while loop, not just an if, to find the truly minimal window at each valid point.
- Forgetting the required distinct-character count check (`need.size()`) rather than a total character count check.
- Returning an empty string incorrectly when `bestLen` was never updated (must check `bestLen == INT_MAX`, not `bestLen == 0`).
- Not handling duplicate characters in `t` correctly (e.g., `t='AABC'` needs TWO A's, not one).

INTERVIEW TIPS

- Explicitly explain the 'formed' counter trick before coding — this is THE signal that separates senior candidates from those who brute-force compare maps.
- State the window invariant clearly: 'expand right to find validity, then shrink left maximally while it stays valid, recording the best window at each valid point.'
- Mention this problem is widely considered one of the hardest 'medium-labeled' sliding window problems — acknowledging difficulty shows calibrated confidence, not arrogance.
- Draw the need/window/formed state evolution on the whiteboard for a few steps of the dry run.

FOLLOW-UP QUESTIONS

- How would this change if you needed the window to contain characters in the exact ORDER they appear in `t` (subsequence, not just multiset)?
- How would you extend this to find ALL minimum windows, not just one?
- How would you adapt this for a streaming input where you can't look back arbitrarily?

VARIATIONS

- Easy: Smallest window containing a single required character
- Medium: Permutation in String (LeetCode 567, fixed-size variant using similar formed-counter idea)
- Hard: Minimum Window Substring (LeetCode 76) with duplicate character requirements

PATTERN SUMMARY

Minimum Window Substring is sliding window's final boss — the $O(1)$ 'formed' counter trick, not full map comparison, is what makes the $O(n)$ solution actually $O(n)$.

PATTERN 31

Subsequence Problems

FAANG Frequency	High
LeetCode Importance	High
Codeforces Importance	Medium
Companies Asking	Amazon, Google, Meta, Microsoft
Difficulty	Medium
Time Complexity	$O(n)$ to $O(n*m)$
Space Complexity	$O(1)$ to $O(n*m)$
Problems Covered	Is Subsequence, Number of Matching Subsequences, Distinct Subsequences

INTUITION

- Subsequence problems allow SKIPPING characters while preserving relative order — the two-pointer check (Pattern 04) handles simple existence questions, while COUNTING distinct subsequences requires DP since multiple skip-choices can lead to the same result.
- The key distinction to always state upfront: subsequence $\neq$ substring. This single clarification prevents an entire class of miscommunication in interviews.

PATTERN RECOGNITION — Trigger Words

- subsequence (not necessarily contiguous)
- count distinct subsequences
- does target appear as an ordered subset of characters

If interviewer says “how many distinct subsequences of s equal t ” — think 2D DP counting ways, not a simple two-pointer boolean check.

DRY RUN

```
s="rabbbit", t="rabbit"
dp[i][j] = number of ways s[0..i) forms t[0..j)
multiple 'b's in s create multiple ways to match – dp accumulates these, giving final answer 3
.
```

BRUTE FORCE

- Recursively try including/excluding each character of *s* without memoization — exponential.

Time: $O(2^n)$ | Space: $O(n)$ recursion depth

BETTER SOLUTION

- Top-down memoized recursion over (i, j) indices into *s* and *t*.

Time: $O(n \cdot m)$ | Space: $O(n \cdot m)$

OPTIMAL SOLUTION

- Bottom-up 2D DP: $dp[i][j] = dp[i-1][j]$ (skip $s[i-1]$) + $dp[i-1][j-1]$ if $s[i-1] == t[j-1]$ (use it to match).

Time: $O(n \cdot m)$ | Space: $O(m)$ with rolling row

C++ CODE (Optimal)

`pattern31_optimal.cpp`

```
int numDistinct(string s, string t) {
    int n = s.size(), m = t.size();
    vector<unsigned long long> dp(m + 1, 0);
    dp[0] = 1; // empty t matches in exactly one way
    for (int i = 1; i <= n; i++) {
        for (int j = m; j >= 1; j--) {
            if (s[i - 1] == t[j - 1]) dp[j] += dp[j - 1];
        }
    }
    return (int)dp[m];
}
```

COMMON MISTAKES

- Confusing this with simple 'is subsequence' existence check — counting DISTINCT ways requires a completely different DP formulation.
- Iterating j in the wrong direction when space-optimizing to 1D (must go right-to-left to avoid using already-updated values from the same iteration).
- Forgetting $dp[0] = 1$ as the base case (empty target string matches in exactly one way — the empty match).
- Integer overflow: counting distinct subsequences can grow very large, requiring long long / unsigned long long.
- Not recognizing when a simple two-pointer suffices (existence check) vs when full DP is required (counting).
- Off-by-one between string indices and DP indices.
- Assuming greedy 'always match earliest occurrence' works for counting problems — it doesn't, since ALL possible matches must be counted, not just one.
- For 'Number of Matching Subsequences' (many words against one string), not precomputing per-character next-occurrence positions to avoid $O(n)$ per word.

INTERVIEW TIPS

- Immediately distinguish existence-check (two-pointer, $O(n)$) from counting (DP, $O(n*m)$) — ask the interviewer which is needed if ambiguous.
- For 'Number of Matching Subsequences' at scale (many words), mention bucketing words by their next needed character for an $O(n + \text{total length})$ sweep — a well-known optimization.
- Mention overflow risk explicitly when counting subsequences — shows attention to real implementation details.
- State the space-optimization direction rule out loud (right-to-left) to preempt a subtle bug.

FOLLOW-UP QUESTIONS

- How would you solve 'Number of Matching Subsequences' efficiently for thousands of words against one string?
- How would you modify this DP to return one example of a matching subsequence, not just the count?
- What's the overflow-safe way to return counts that could exceed 64-bit integers?

VARIATIONS

- Easy: Is Subsequence (LeetCode 392)
- Medium: Number of Matching Subsequences (LeetCode 792)
- Hard: Distinct Subsequences (LeetCode 115) with duplicate-heavy strings

PATTERN SUMMARY

Subsequence problems split into cheap existence checks (two-pointer) and expensive counting problems (DP) — know which one is actually being asked.

PATTERN 32

Substring Problems

FAANG Frequency	Very High
LeetCode Importance	Very High
Codeforces Importance	High
Companies Asking	Amazon, Google, Meta, Microsoft, Bloomberg
Difficulty	Medium
Time Complexity	$O(n)$ to $O(n^2)$
Space Complexity	$O(k)$
Problems Covered	Longest Substring Without Repeating Characters, Count Substrings with Only One Distinct Letter, Number of Substrings Containing All Three Characters

INTUITION

- Substring problems require CONTIGUOUS runs — this is the umbrella category that Sliding Window (Pattern 05) and Two Pointer (Pattern 04) both serve; the specific technique depends on whether the constraint is monotonic (window only needs to grow/shrink) or not.
- A common counting trick: 'number of substrings satisfying X ending at each position' summed across all positions, often computed via a sliding window boundary (leftmost valid start) tracked incrementally.

PATTERN RECOGNITION — Trigger Words

- contiguous substring satisfying a condition
- count all substrings with property X
- substring window constraints

If interviewer says "count the number of substrings containing at least one 'a', one 'b', and one 'c'" — think sliding window tracking the leftmost valid start, summing contributions per right pointer.

DRY RUN

```
s="abcabc", count substrings containing a,b,c all at least once
for each right pointer, find the leftmost valid left; every start from 0 to that left also works -> add (left+1) to the count
```

BRUTE FORCE

- Check every substring directly for the property — $O(n^2)$ substrings, $O(n)$ check each.

Time: $O(n^3)$ | Space: $O(1)$

BETTER SOLUTION

- Precompute prefix structures (Pattern 12) to check the property in $O(1)$ per substring, still enumerating $O(n^2)$ substrings.

Time: $O(n^2)$ | Space: $O(n)$

OPTIMAL SOLUTION

- Sliding window tracking the boundary of validity; for counting variants, use the 'leftmost valid start contributes (left+1) substrings ending here' trick to sum in $O(n)$ total.

Time: $O(n)$ | Space: $O(k)$

C++ CODE (Optimal)

pattern32_optimal.cpp

```
int numberOfSubstrings(string s) {
    vector<int> lastSeen(3, -1);
    int count = 0;
    for (int right = 0; right < (int)s.size(); right++) {
        lastSeen[s[right] - 'a'] = right;
        int minLast = min({lastSeen[0], lastSeen[1], lastSeen[2]});
        if (minLast != -1) count += minLast + 1; // all substrings starting at 0..minLast qual
    }
    return count;
}
```

COMMON MISTAKES

- Trying to enumerate every substring explicitly for a counting problem when a per-right-pointer contribution trick gives $O(n)$.
- Off-by-one in the 'contributes (left+1) substrings' logic — easy to be one off in either direction.
- Confusing substring (contiguous) counting tricks with subsequence (Pattern 31) counting DP — different problems, different techniques entirely.
- Not tracking the RIGHTMOST invalid boundary correctly, leading to double counting or undercounting.
- Assuming a simple two-pointer suffices when the condition isn't monotonic (i.e., growing the window doesn't monotonically preserve/break validity) — verify monotonicity before applying the trick.
- Ignoring integer overflow for very long strings with $O(n^2)$ possible substrings (use long long for the count).
- Not clarifying whether overlapping substrings should be counted separately (usually yes, unless stated otherwise).
- Forgetting to reset lastSeen/window state between independent test cases.

INTERVIEW TIPS

- Explicitly verify the monotonicity property before applying a sliding-window counting trick: 'as the window grows, does validity only turn on and never flicker off?'
- State the 'contribution' framing clearly: 'for each right pointer, count how many LEFT starting points make this substring valid, and sum those.'
- Mention that substring problems overlap heavily with sliding window (Pattern 05); name the specific optimization (contribution counting, prefix precompute) you're applying.
- Use long long for counts on large inputs to avoid silent overflow bugs.

FOLLOW-UP QUESTIONS

- How would you count substrings with EXACTLY k distinct characters (using $\text{atMost}(k) - \text{atMost}(k-1)$)?
- How would you adapt the contribution-counting trick for a different validity condition?
- What if the alphabet were much larger — does the same technique still scale?

VARIATIONS

- Easy: Count Substrings with Only One Distinct Letter
- Medium: Number of Substrings Containing All Three Characters (LeetCode 1358)
- Hard: Count of substrings with exactly K distinct characters using $\text{atMost}(K) - \text{atMost}(K-1)$

PATTERN SUMMARY

Substring counting problems often reduce to a per-position 'contribution' sum via sliding window — verify monotonicity first, then count instead of enumerate.

PATTERN 33

Lexicographical Problems

FAANG Frequency	Medium
LeetCode Importance	Medium
Codeforces Importance	Medium
Companies Asking	Google, Amazon, Adobe
Difficulty	Medium
Time Complexity	$O(n)$ to $O(n \log n)$
Space Complexity	$O(n)$
Problems Covered	Remove Duplicate Letters, Smallest Subsequence of Distinct Characters, Next Greater Element style string problems

INTUITION

- Lexicographically-smallest/largest problems usually combine a GREEDY choice (pick the smallest possible character now) with a MONOTONIC STACK to allow 'undoing' a previous greedy choice if a better option appears later and the previous character can still appear again.
- The critical enabling condition: you can only pop a character off your result-so-far if that character still appears LATER in the string (otherwise you'd lose it permanently) — this requires tracking last-occurrence indices.

PATTERN RECOGNITION — Trigger Words

- smallest/largest string in lexicographic order
- remove duplicate letters keeping smallest result
- greedy character selection with undo

If interviewer says "remove duplicate letters so the result is the smallest possible in lexicographic order, keeping one of each" — think monotonic stack + last-occurrence tracking.

DRY RUN

```
s="cbacdcbc"
lastOcc: c->7, b->6, a->2, d->4
build stack greedily: pop 'c' if 'b' is smaller AND 'c' appears again later -> continue until
stack is monotonic and valid
```

BRUTE FORCE

- Try all valid subsequences containing each distinct character exactly once, compare lexicographically — factorial/exponential blowup.

Time: $O(2^n)$ or worse | Space: $O(n)$

BETTER SOLUTION

- Greedy without the 'reappears later' safety check — can incorrectly discard a character that never appears again, producing an invalid result.

Time: $O(n)$ | Space: $O(n)$

OPTIMAL SOLUTION

- Monotonic stack: maintain last-occurrence index per character; while the stack top is greater than the current character AND the top can still appear later, pop it; push current character if not already in the stack.

Time: $O(n)$ | Space: $O(k)$ alphabet size

C++ CODE (Optimal)

pattern33_optimal.cpp

```
string removeDuplicateLetters(string s) {
    vector<int> lastOcc(26, 0);
    for (int i = 0; i < (int)s.size(); i++) lastOcc[s[i] - 'a'] = i;
    vector<bool> inStack(26, false);
    string stack;
    for (int i = 0; i < (int)s.size(); i++) {
        char c = s[i];
        if (inStack[c - 'a']) continue;
        while (!stack.empty() && stack.back() > c && lastOcc[stack.back() - 'a'] > i) {
            inStack[stack.back() - 'a'] = false;
            stack.pop_back();
        }
        stack.push_back(c);
        inStack[c - 'a'] = true;
    }
    return stack;
}
```

COMMON MISTAKES

- Popping a character off the stack without checking it reappears later — permanently losing a required character.
- Forgetting the `inStack` tracking, allowing the same character to be pushed multiple times.
- Using `stack.back() > c` with a strict/non-strict comparison mismatch (must be strictly greater to preserve stability for equal characters).
- Not precomputing `lastOcc` for $O(1)$ lookups, instead scanning ahead each time (turns $O(n)$ into $O(n^2)$).
- Confusing 'smallest lexicographic result keeping all distinct characters' with simple sorting (sorting ignores the ORIGINAL relative order constraint entirely).
- Off-by-one when computing `lastOcc` (must reflect the LAST index, requiring a full scan first, not the first-seen index).
- Not handling repeated greedy-undo cycles correctly when multiple characters could be popped in sequence.
- Ignoring that this problem is a specific case of the broader 'monotonic stack for lexicographic optimization' pattern that also applies to problems like Next Greater Element.

INTERVIEW TIPS

- Explain the two preconditions for popping clearly: '(1) the top is lexicographically larger, AND (2) it reappears later so removing it now is safe.'
- Precompute `lastOcc` for $O(1)$ lookups — mention this explicitly as the optimization that keeps the algorithm $O(n)$ instead of $O(n^2)$.
- Draw the stack state evolution step-by-step for a tricky example with multiple pop-cycles.
- Connect this to the broader monotonic-stack pattern (Pattern 35) — shows you see the structural similarity across problems.

FOLLOW-UP QUESTIONS

- How would you generalize this to keep the K smallest lexicographic result with some other constraint (e.g., 'Smallest Subsequence of Distinct Characters' with a length constraint)?
- What changes if you wanted the LARGEST lexicographic result instead of smallest?
- How does this relate to the classic 'Remove K Digits to form smallest number' problem?

VARIATIONS

- Easy: Smallest Subsequence of Distinct Characters (LeetCode 316, same core idea)
- Medium: Remove Duplicate Letters (LeetCode 316/1081)
- Hard: Remove K Digits (LeetCode 402) combined with lexicographic monotonic stack reasoning

PATTERN SUMMARY

Lexicographic optimization problems combine greedy character choice with a monotonic stack that can safely 'undo' using last-occurrence tracking.

PATTERN 34

Greedy String Problems

FAANG Frequency	Medium
LeetCode Importance	Medium
Codeforces Importance	Medium
Companies Asking	Amazon, Google, Adobe, Flipkart
Difficulty	Medium
Time Complexity	$O(n)$ to $O(n \log n)$
Space Complexity	$O(1)$ to $O(n)$
Problems Covered	Partition Labels, Reorganize String, Task Scheduler (string-adjacent), Merge Strings Alternately

INTUITION

- Greedy string problems require proving a LOCAL choice (e.g., 'extend the current partition as far as needed') leads to a GLOBAL optimum — the hard part is proving the greedy choice is safe, not implementing it.
- A recurring greedy sub-technique: track the LAST occurrence of every character first, then greedily extend a window/partition until it covers the last occurrence of every character currently inside it.

PATTERN RECOGNITION — Trigger Words

- partition the string into as many parts as possible
- each character appears in only one part
- rearrange so no two adjacent are the same

If interviewer says "partition the string so each letter appears in at most one part, maximize the number of parts" — think greedy: extend the current partition to the last occurrence of every character seen so far.

DRY RUN

```
s="ababcbacadefegdehijhklj"
lastOcc precomputed for every character
start partition at 0, extend 'end' to max(lastOcc[c]) for each c seen; when i==end, cut a part
ition here
```

BRUTE FORCE

- Try every possible partitioning and check validity — exponential number of partitions to consider.

Time: $O(2^n)$ | Space: $O(n)$

BETTER SOLUTION

- For each starting position, scan ahead to find where a valid partition could end — redundant re-scanning across partitions.

Time: $O(n^2)$ | Space: $O(n)$

OPTIMAL SOLUTION

- Precompute lastOcc for all characters in $O(n)$, then single greedy pass extending 'end' to the max lastOcc seen, cutting whenever the current index reaches 'end'.

Time: $O(n)$ | Space: $O(1)$ fixed alphabet

C++ CODE (Optimal)

pattern34_optimal.cpp

```
vector<int> partitionLabels(string s) {
    vector<int> lastOcc(26, 0);
    for (int i = 0; i < (int)s.size(); i++) lastOcc[s[i] - 'a'] = i;
    vector<int> result;
    int start = 0, end = 0;
    for (int i = 0; i < (int)s.size(); i++) {
        end = max(end, lastOcc[s[i] - 'a']);
        if (i == end) {
            result.push_back(end - start + 1);
            start = i + 1;
        }
    }
    return result;
}
```

COMMON MISTAKES

- Not precomputing lastOcc first, instead scanning ahead per-partition (turns $O(n)$ into $O(n^2)$).
- Forgetting to update 'end' using max() — a later character's last occurrence might extend the partition further than expected.
- Cutting a partition too early, before ALL characters currently inside the window have had their last occurrence covered.
- Confusing this greedy interval-extension idea with a completely different technique like sliding window with shrinking (this window NEVER shrinks, only extends).
- Not proving/stating why greedy works here: every character's FIRST and LAST occurrence must be in the same partition, so extending to cover last occurrences is provably necessary and sufficient.
- Assuming greedy works for ALL string partition problems without verifying the specific proof for each one (greedy is not universally safe).
- Off-by-one in partition length calculation (end - start + 1).
- For Reorganize String / Task Scheduler style problems, forgetting to use a max-heap by frequency instead of this last-occurrence technique — different greedy sub-pattern entirely.

INTERVIEW TIPS

- Always state the greedy proof sketch, even briefly: 'since every character's occurrences must stay together, extending to the last occurrence is both necessary and sufficient.'
- Distinguish this specific greedy sub-pattern (extend-to-last-occurrence) from the max-heap-by-frequency sub-pattern (Reorganize String, Task Scheduler) — they look similar but solve different constraints.
- Precompute whatever lookup (last occurrence, frequency) the greedy choice depends on BEFORE the main greedy pass.
- Be ready to explain why a greedy approach does NOT work for a superficially similar problem, if asked — shows critical thinking, not pattern-matching blindly.

FOLLOW-UP QUESTIONS

- How would 'Reorganize String' need a different greedy approach (max-heap by frequency) than Partition Labels?
- Can you prove why the extend-to-last-occurrence greedy choice is always optimal here?
- How would this change if partitions needed a MINIMUM size constraint too?

VARIATIONS

- Easy: Merge Strings Alternately (LeetCode 1768, simple greedy zip)
- Medium: Partition Labels (LeetCode 763)
- Hard: Reorganize String / Task Scheduler style greedy with max-heap by frequency

PATTERN SUMMARY

Greedy string problems require proving local choices are globally safe — extend-to-last-occurrence and max-heap-by-frequency are the two dominant greedy sub-patterns.

PATTERN 35

Stack Based String Problems

FAANG Frequency	Very High
LeetCode Importance	Very High
Codeforces Importance	Medium
Companies Asking	Amazon, Google, Meta, Microsoft, Adobe
Difficulty	Medium
Time Complexity	$O(n)$
Space Complexity	$O(n)$
Problems Covered	Valid Parentheses, Decode String, Basic Calculator, Remove All Adjacent Duplicates

INTUITION

- Whenever a problem involves NESTED structure (parentheses, brackets, nested encodings like '3[a2[c]]') or requires 'undo the most recent thing' behavior (removing adjacent duplicates, backspace simulation), a stack is the natural data structure — it mirrors the LIFO nature of nesting.
- For nested decoding problems specifically, the stack typically holds PARTIAL results (current string being built) and their associated repeat counts, pushed when entering a bracket and popped/combined when exiting one.

PATTERN RECOGNITION — Trigger Words

- nested brackets/parentheses
- decode a run-length-encoded nested string
- remove adjacent duplicates
- evaluate an expression with parentheses

If interviewer says "decode a string like '3[a2[c]]' into 'accaccacc'" — think stack holding (previous string, repeat count) pairs, pushed on '[' and combined on ']'.

DRY RUN

```
s="3[a2[c]]"
push ("",3) on '3['
push ("a",2) on 'a2['
see 'c', build "c"
on ']': pop ("a",2) -> combine: "a" + "c"*2 = "acc"
on ']': pop ("",3) -> combine: "" + "acc"*3 = "accaccacc"
```

BRUTE FORCE

- Try to parse nested structure with recursive string manipulation and heavy substring slicing — correct but often $O(n^2)$ due to repeated string concatenation/copying.

Time: $O(n^2)$ with naive string ops | Space: $O(n)$

BETTER SOLUTION

- Recursive descent parsing (treat brackets as recursive calls) — clean and correct, but uses call-stack space implicitly instead of an explicit stack.

Time: $O(n)$ | Space: $O(n)$ call stack

OPTIMAL SOLUTION

- Explicit stack-based iterative parsing: push (currentString, currentCount) on '[', pop and combine on ']', avoiding recursion overhead and making the state transitions explicit and debuggable.

Time: $O(n)$ | Space: $O(n)$

C++ CODE (Optimal)

pattern35_optimal.cpp

```
string decodeString(string s) {
    stack<pair<string,int>> st;
    string current;
    int num = 0;
    for (char c : s) {
        if (isdigit(c)) {
            num = num * 10 + (c - '0');
        } else if (c == '[') {
            st.push({current, num});
            current = "";
            num = 0;
        } else if (c == ']') {
            auto [prevStr, count] = st.top(); st.pop();
            string expanded;
            for (int i = 0; i < count; i++) expanded += current;
            current = prevStr + expanded;
        } else {
            current += c;
        }
    }
    return current;
}
```

COMMON MISTAKES

- Forgetting multi-digit numbers (e.g., '12[a]') by not accumulating $\text{num} = \text{num} * 10 + \text{digit}$ across multiple characters.
- Not resetting `num` and current appropriately when entering a new bracket level.
- Using recursion without a clear base case, risking stack overflow on deeply nested inputs (mention iterative stack as the safer production choice).
- Confusing WHEN to push (on '[') vs WHEN to pop-and-combine (on ']') — easy to get the direction backward under interview pressure.
- Inefficient string concatenation inside the expansion loop for very large repeat counts (consider `reserve()` or building smarter for huge counts).
- Not handling malformed input gracefully if the problem doesn't guarantee well-formed brackets (usually guaranteed in interview versions, but worth mentioning).
- For Valid Parentheses style problems, forgetting to check the stack is EMPTY at the end (unmatched opening brackets remain).
- Popping from an empty stack without a guard check, causing undefined behavior/crash on malformed input.

INTERVIEW TIPS

- State explicitly: 'nested structure means LIFO behavior, so a stack naturally mirrors the recursive structure without needing actual recursion.'
- Walk through the push/pop timing carefully: push happens on opening bracket (saving state), pop-and-combine happens on closing bracket (restoring and merging state).
- Mention the recursive-descent alternative and its tradeoff (implicit call stack depth risk vs explicit stack clarity).
- For calculator/expression problems, mention handling operator precedence via a stack of pending operations, a related but distinct stack sub-pattern.

FOLLOW-UP QUESTIONS

- How would you handle a Basic Calculator problem with `+`, `-`, `*`, `/` and parentheses using a similar stack approach?
- What's the maximum recursion depth risk here, and how does the iterative stack version avoid it?
- How would you validate malformed input gracefully instead of assuming well-formed brackets?

VARIATIONS

- Easy: Valid Parentheses (LeetCode 20)
- Medium: Decode String (LeetCode 394)
- Hard: Basic Calculator (LeetCode 224) with nested parentheses and all four operators

PATTERN SUMMARY

Nested string structure = stack territory. Push state on entering a nested level, pop-and-combine on exit — the universal template for brackets, decoding, and expression evaluation.

PATTERN 36

Deque Based String Problems

FAANG Frequency	Low-Medium
LeetCode Importance	Medium
Codeforces Importance	Medium
Companies Asking	Google, Amazon, Adobe
Difficulty	Medium
Time Complexity	$O(n)$
Space Complexity	$O(k)$ window size
Problems Covered	Sliding Window Maximum (character-code variant), Shortest Subarray-style constraints on strings

INTUITION

- A deque (double-ended queue) is the right tool when you need to efficiently maintain a MONOTONIC property (max or min) over a SLIDING WINDOW, supporting $O(1)$ amortized push/pop from both ends — useful when string problems are recast as sliding-window-over-character-codes problems.
- The core technique: maintain indices in the deque such that their corresponding values are monotonically decreasing (for max) or increasing (for min); pop from the back whenever a new element would break monotonicity, and pop from the front when the front falls outside the current window.

PATTERN RECOGNITION — Trigger Words

- sliding window maximum/minimum
- maintain a monotonic property as a window slides
- longest window where max-min constraint holds

If interviewer recasts a string problem as 'find the longest window where the difference between the most and least frequent (or highest/lowest ASCII) character is bounded' — think two monotonic deques (one for max, one for min).

DRY RUN

```

window sliding over character codes [4,2,7,1,5]
maxDeque maintains decreasing order: pushing 7 pops 2 and 4 first (they can never be the max a
gain)
minDeque maintains increasing order similarly

```

BRUTE FORCE

- For every window, scan all elements to find max/min directly — $O(k)$ per window.

Time: $O(n*k)$ | Space: $O(1)$

BETTER SOLUTION

- Use a balanced BST / ordered multiset to track max/min per window in $O(\log k)$ per operation.

Time: $O(n \log k)$ | Space: $O(k)$

OPTIMAL SOLUTION

- Two monotonic deques (one decreasing for max, one increasing for min), each element pushed and popped at most once — true $O(1)$ amortized per window slide.

Time: $O(n)$ | Space: $O(k)$

C++ CODE (Optimal)

pattern36_optimal.cpp

```
// Longest substring where max(charcode) - min(charcode) <= limit, sliding window + 2 monotonic deques
int longestSubarray(vector<int>& codes, int limit) {
    deque<int> maxDQ, minDQ; // store indices
    int left = 0, best = 0;
    for (int right = 0; right < (int)codes.size(); right++) {
        while (!maxDQ.empty() && codes[maxDQ.back()] <= codes[right]) maxDQ.pop_back();
        maxDQ.push_back(right);
        while (!minDQ.empty() && codes[minDQ.back()] >= codes[right]) minDQ.pop_back();
        minDQ.push_back(right);
        while (codes[maxDQ.front()] - codes[minDQ.front()] > limit) {
            left++;
            if (maxDQ.front() < left) maxDQ.pop_front();
            if (minDQ.front() < left) minDQ.pop_front();
        }
        best = max(best, right - left + 1);
    }
    return best;
}
```

COMMON MISTAKES

- Storing VALUES in the deque instead of INDICES, making it impossible to know when an element has fallen outside the window.
- Getting the monotonicity direction backward (max deque must stay decreasing; min deque must stay increasing).
- Popping from the front unconditionally instead of checking if the front index is actually outside the current window.
- Using a single deque when the problem genuinely requires tracking BOTH max and min simultaneously (needs two separate deques).
- Not recognizing when a simpler two-pointer/frequency-array approach (Pattern 05) would suffice instead of the heavier deque machinery.
- Off-by-one in window boundary checks (front index $< \text{left}$ vs $\leq \text{left}$).
- Assuming deque operations are always $O(1)$ worst-case per single operation — they're $O(1)$ AMORTIZED across the whole run, individual operations can still pop multiple elements.
- Forgetting to shrink the window (advance left) inside a while loop, not just an if, to maintain the constraint maximally.

INTERVIEW TIPS

- Clarify explicitly why a deque beats brute-force here: 'each index is pushed and popped at most once across the whole algorithm, giving amortized $O(1)$ per step, $O(n)$ total.'
- State the monotonicity invariant for each deque clearly before coding: 'front of maxDQ is always the current window's maximum.'
- Compare against the ordered-multiset $O(n \log k)$ alternative and explain why deque is asymptotically better when applicable.
- Recognize and mention when this pattern is overkill — many string problems only need Pattern 05's simpler frequency-array sliding window.

FOLLOW-UP QUESTIONS

- How would you adapt this if the constraint were on frequency counts instead of raw character codes?
- What's the difference between this and using an ordered multiset/balanced BST for the same problem?
- Can you prove the amortized $O(1)$ claim for deque push/pop operations across the full run?

VARIATIONS

- Easy: Sliding Window Maximum (LeetCode 239, classic array version)
- Medium: Longest Substring with character-code range constraint
- Hard: Combining monotonic deque with additional frequency constraints for compound conditions

PATTERN SUMMARY

Dequeues maintain monotonic max/min over a sliding window in amortized $O(1)$ — reach for this only when simpler frequency-array sliding window (Pattern 05) genuinely isn't enough.

PATTERN 37

Binary Search on Answer + String

FAANG Frequency	Medium
LeetCode Importance	Medium
Codeforces Importance	High
Companies Asking	Google, Amazon, Databricks
Difficulty	Hard
Time Complexity	$O(n \log n)$
Space Complexity	$O(n)$
Problems Covered	Longest Duplicate Substring, Minimize Max Distance style string problems, Split string into k parts minimizing max length

INTUITION

- When a problem asks for the OPTIMAL VALUE of some length/count and 'can we achieve length k?' is easy to check independently (monotonic feasibility), binary search on that answer value transforms an otherwise hard optimization into repeated feasibility checks.
- For strings specifically, this combines beautifully with rolling hash (Pattern 13): binary search on substring length, and for each candidate length, use rolling hash + hashset to check in $O(n)$ if any length-k substring repeats.

PATTERN RECOGNITION — Trigger Words

- longest/shortest X such that condition Y holds
- minimize the maximum / maximize the minimum
- feasibility of a length is monotonic (works for k, works for all $k' < k$)

If interviewer says "find the length of the longest duplicated substring" — think binary search on length + rolling hash feasibility check at each candidate length, rather than checking every length from n down to 1 directly.

DRY RUN

```
s of length n, binary search bounds [0, n-1] on candidate duplicate-length k
check(k): roll hash all length-k substrings, use hashset to detect a repeat -> true/false
feasible k's are monotonic (if length k works, so does any smaller length) -> binary search applies
```

BRUTE FORCE

- Try every possible length from $n-1$ down to 1, checking each with an $O(n)$ or $O(n \log n)$ feasibility check, stopping at the first success — $O(n^2)$ worst case overall.

Time: $O(n^2)$ | Space: $O(n)$

BETTER SOLUTION

- Binary search on length, but use a naive $O(n^2)$ substring comparison for the feasibility check at each length — correct but the inner check dominates cost.

Time: $O(n^2 \log n)$ | Space: $O(n)$

OPTIMAL SOLUTION

- Binary search on length ($O(\log n)$ iterations), each feasibility check done via rolling hash in $O(n)$ — total $O(n \log n)$.

Time: $O(n \log n)$ | Space: $O(n)$

C++ CODE (Optimal)

pattern37_optimal.cpp

```
// Skeleton: binary search on length, check() uses rolling hash for O(n) feasibility
bool check(const string &s, int len) {
    unordered_set<long long> seen;
    long long base = 131, mod = 1e9 + 7, h = 0, power = 1;
    for (int i = 0; i < len; i++) power = (power * base) % mod;
    for (int i = 0; i < (int)s.size(); i++) {
        h = (h * base + s[i]) % mod;
        if (i >= len) h = (h - (s[i - len] * power) % mod * base % mod + (long long)mod*mod) %
mod;
        if (i >= len - 1) { if (!seen.insert(h).second) return true; }
    }
    return false;
}

int longestRepeatedLength(const string &s) {
    int lo = 1, hi = (int)s.size() - 1, ans = 0;
    while (lo <= hi) {
        int mid = lo + (hi - lo) / 2;
        if (check(s, mid)) { ans = mid; lo = mid + 1; }
        else hi = mid - 1;
    }
    return ans;
}
```

COMMON MISTAKES

- Failing to first prove/verify the MONOTONICITY of feasibility — binary search on answer is only valid when 'works for k' implies 'works for all smaller (or larger) k' consistently in one direction.
- Using an $O(n^2)$ feasibility check inside the binary search, negating the benefit of the $\log n$ factor (should combine with rolling hash for $O(n)$ per check).
- Off-by-one in binary search bounds (lo/hi/mid updates) — extremely common bug, always dry-run on a small example.
- Not verifying hash matches with direct string comparison in the feasibility check when hash collisions are a real risk (see Pattern 13's mistakes).
- Confusing 'binary search on the ANSWER' (searching over possible output values) with 'binary search on the ARRAY/STRING' (searching over positions) — fundamentally different search spaces.
- Not handling the edge case where no valid length exists (e.g., no duplicated substring at all).
- Assuming this pattern applies to every 'longest/shortest X' problem without first confirming the monotonicity precondition.
- Recomputing the rolling hash base powers inside the feasibility check repeatedly instead of once per check call (still fine complexity-wise, but worth being precise about).

INTERVIEW TIPS

- Always state the monotonicity argument explicitly before applying binary search on answer: 'if a duplicate of length k exists, one of length k-1 must also exist (as a substring of it) — so feasibility is monotonic.'
- Explicitly combine this with rolling hash and mention the double-hashing safety consideration from Pattern 13.
- Draw the binary search bounds and a couple of mid-point iterations to demonstrate correctness of the search boundaries.
- Compare this approach's $O(n \log n)$ against the suffix-array-based $O(n \log n)$ alternative (Pattern 11) — both valid, different constant factors and implementation complexity.

FOLLOW-UP QUESTIONS

- How would you prove the monotonicity property required for binary search to be valid here?
- How does this approach compare to using a Suffix Array + LCP array for the same problem?
- What if you needed ALL longest duplicated substrings, not just the length of one?

VARIATIONS

- Easy: Binary search on a simple monotonic string-length feasibility check
- Medium: Split a string into k parts minimizing the maximum part length
- Hard: Longest Duplicate Substring (LeetCode 1044) via binary search + rolling hash

PATTERN SUMMARY

Binary search on answer applies when feasibility of a length is monotonic — combined with rolling hash, it turns 'longest duplicated substring' into an elegant $O(n \log n)$.

PATTERN 38

Bit Manipulation + String

FAANG Frequency	Medium
LeetCode Importance	Medium
Codeforces Importance	High
Companies Asking	Google, Amazon, Microsoft
Difficulty	Medium
Time Complexity	$O(n)$
Space Complexity	$O(1)$
Problems Covered	Maximum Product of Word Lengths (bitmask), Find the Longest Substring Containing Vowels in Even Counts

INTUITION

- When a string problem only cares about the PRESENCE or PARITY (even/odd count) of characters from a small fixed alphabet (like 26 lowercase letters or 5 vowels), encoding each string/prefix as a BITMASK turns character-set comparisons into $O(1)$ integer operations (AND, XOR, popcount) instead of $O(26)$ array comparisons.
- XOR-based parity tracking is especially powerful for 'even count of each character' problems: XOR-ing a bit on for each character occurrence naturally toggles it back to 0 whenever that character's count becomes even again.

PATTERN RECOGNITION — Trigger Words

- fixed small alphabet (like 26 letters or vowels)
- no common characters between two words
- even/odd count of each character in a substring

If interviewer says "find two words in an array with no common letters that maximize the product of lengths" — think bitmask per word, compare with $AND == 0$ in $O(1)$ instead of $O(26)$ per pair.

DRY RUN

```
word="abc" -> bitmask = (1<<0)|(1<<1)|(1<<2) = 0b111
two words share no letters iff (mask1 & mask2) == 0 -- O(1) check instead of scanning both words' character sets
```

BRUTE FORCE

- For every pair of words, compare their character sets directly using a set intersection check — $O(k)$ per pair.

Time: $O(n^2 * k)$ | Space: $O(n*k)$

BETTER SOLUTION

- Precompute a boolean array of 26 flags per word, compare arrays per pair — still $O(26)$ per pair instead of $O(1)$.

Time: $O(n^2 * 26)$ | Space: $O(n*26)$

OPTIMAL SOLUTION

- Precompute a 26-bit bitmask per word in $O(k)$ each, then compare pairs via a single AND operation in $O(1)$.

Time: $O(n*k + n^2)$ | Space: $O(n)$

C++ CODE (Optimal)

`pattern38_optimal.cpp`

```
int maxProduct(vector<string>& words) {
    int n = words.size();
    vector<int> masks(n, 0);
    for (int i = 0; i < n; i++)
        for (char c : words[i]) masks[i] |= (1 << (c - 'a'));
    int best = 0;
    for (int i = 0; i < n; i++)
        for (int j = i + 1; j < n; j++)
            if ((masks[i] & masks[j]) == 0)
                best = max(best, (int)(words[i].size() * words[j].size()));
    return best;
}
```

COMMON MISTAKES

- Forgetting that bitmask encodes PRESENCE only, losing frequency information — wrong choice if the problem actually cares about counts, not just which characters appear.
- Using AND when the problem actually needs XOR-based parity tracking (even/odd counts), or vice versa — these solve different questions.
- Off-by-one in bit shift amounts ($1 \ll (c - 'a')$ for lowercase, different offset for uppercase or digits).
- Not precomputing masks once per word/prefix, instead recomputing inside the $O(n^2)$ comparison loop.
- Assuming bitmask tricks apply to large or Unicode alphabets — only practical for small fixed alphabets (26, 52, a handful of vowels) that fit in a machine word.
- For 'even count' style problems, forgetting that a prefix-XOR-mask approach requires comparing EQUAL masks at two prefix positions (same trick as prefix-sum-equal-subarray problems), not just checking a single mask.
- Using `__builtin_popcount` incorrectly or not being aware of it as a fast way to count set bits when needed.
- Not clarifying whether case sensitivity affects which bit positions are used.

INTERVIEW TIPS

- State explicitly what the bitmask ENCODES (presence vs parity) before deciding AND/OR/XOR operations — this single clarification prevents most bugs in this pattern.
- Mention `__builtin_popcount(mask)` as the fast way to count characters/parity bits when needed.
- For 'no common characters' problems, emphasize the $O(1)$ AND-check as the entire point of the bitmask transformation.
- For 'even count' substring problems, connect explicitly to the prefix-XOR-equal-subarray technique from prefix sum problems — shows pattern transfer across domains.

FOLLOW-UP QUESTIONS

- How would you adapt the bitmask technique for 'even count of each vowel in a substring' style problems (prefix XOR masks)?
- What's the largest alphabet size for which bitmask tricks remain practical (32-bit vs 64-bit integers)?
- How would popcount help if the problem asked for character-count parity instead of presence?

VARIATIONS

- Easy: Check if two strings share any common character using bitmask AND
- Medium: Maximum Product of Word Lengths (LeetCode 318)
- Hard: Find the Longest Substring Containing Vowels in Even Counts (LeetCode 1371) via prefix-XOR bitmask

PATTERN SUMMARY

Bitmasks turn small-fixed-alphabet character-set problems into $O(1)$ integer operations — know whether you need presence (AND/OR) or parity (XOR) before choosing the operation.

PATTERN 39

String Simulation

FAANG Frequency	Medium
LeetCode Importance	Medium
Codeforces Importance	High
Companies Asking	Amazon, Adobe, Microsoft, Goldman Sachs
Difficulty	Easy-Medium
Time Complexity	$O(n)$ to $O(n^2)$
Space Complexity	$O(n)$
Problems Covered	Simplify Path, Text Justification, Zigzag Conversion, String Multiplication

INTUITION

- Simulation problems don't have a clever algorithmic trick — they test whether you can precisely translate a set of REAL-WORLD RULES (like Unix path resolution, or text justification spacing rules) into careful, bug-free step-by-step code.
- The skill being tested is disciplined state-machine thinking: identify every distinct 'mode' or 'case' the rules describe, and handle each explicitly rather than trying to find a shortcut formula.

PATTERN RECOGNITION — Trigger Words

- simulate a real-world process (paths, justification, multiplication)
- step-by-step rules with several edge cases
- no obvious algorithmic trick, just careful implementation

If interviewer says “simplify this Unix-style file path” — think stack-based simulation: split by '/', push real directory names, pop on '.', ignore '.' and empty segments.

DRY RUN

```
path="/a/./b/../../../../c/"
split by '/': [a, ., b, .., .., c]
process: push 'a' -> ignore '.' -> push 'b' -> pop (b) on '..' -> pop (a) on '..' -> push 'c'
result: "/c"
```

BRUTE FORCE

- Try to use regex substitution repeatedly to collapse './' and 'x/./' patterns — fragile, hard to get fully correct across all edge cases, and can be slow with repeated passes.

Time: $O(n^2)$ with repeated regex passes | Space: $O(n)$

BETTER SOLUTION

- Manual character-by-character parsing without a clear stack abstraction — correct if done carefully, but bug-prone and hard to reason about.

Time: $O(n)$ | Space: $O(n)$

OPTIMAL SOLUTION

- Split the path into tokens by '/', use an explicit stack: push valid directory names, pop on '..' (if stack non-empty), skip '.' and empty tokens, then join the stack with '/'.

Time: $O(n)$ | Space: $O(n)$

C++ CODE (Optimal)

pattern39_optimal.cpp

```
string simplifyPath(string path) {
    stack<string> st;
    stringstream ss(path);
    string token;
    while (getline(ss, token, '/')) {
        if (token == "" || token == ".") continue;
        if (token == "..") { if (!st.empty()) st.pop(); }
        else st.push(token);
    }
    vector<string> parts;
    while (!st.empty()) { parts.push_back(st.top()); st.pop(); }
    reverse(parts.begin(), parts.end());
    string result = "/";
    for (int i = 0; i < (int)parts.size(); i++) {
        result += parts[i];
        if (i != (int)parts.size() - 1) result += "/";
    }
    return result;
}
```

COMMON MISTAKES

- Forgetting to skip empty tokens produced by consecutive slashes ('//') during splitting.
- Popping from an empty stack on '..' without a guard check (attempting to go above root should be a no-op, not an error).
- Not handling '.' (current directory, a no-op) distinctly from '..' (parent directory, a pop).
- Manually parsing character-by-character instead of using a clear tokenize-then-process approach, leading to hard-to-debug edge case bugs.
- For Text Justification, miscounting the EXTRA spaces distribution (leftmost gaps get one more space than rightmost when spaces don't divide evenly).
- For Zigzag Conversion, not correctly identifying the direction-change points (top and bottom rows of the zigzag) causing off-by-one row index bugs.
- Not testing edge cases explicitly: root path alone ("/"), trailing slashes, multiple consecutive '..' at the root.
- Rebuilding output with fragile string concatenation instead of using a clear join-with-separator helper, risking extra/missing separators.

INTERVIEW TIPS

- State explicitly that this is a simulation problem with no algorithmic 'trick' — signal that you'll focus on identifying and handling every distinct case carefully.
- For path-style problems, name the ADT explicitly: 'I'll use a stack since parent-directory (..) is naturally a LIFO pop operation.'
- For justification/formatting problems, work out the spacing arithmetic on paper/whiteboard BEFORE coding — these bugs are almost always arithmetic, not logic.
- List out the edge cases you're handling explicitly before or while coding — interviewers reward visible, structured edge-case thinking in simulation problems.

FOLLOW-UP QUESTIONS

- How would you extend Simplify Path to handle Windows-style paths with backslashes and drive letters?
- For Text Justification, how do you correctly distribute uneven extra spaces across gaps?
- How would you handle a MUCH larger path with symbolic links (conceptually, not necessarily code)?

VARIATIONS

- Easy: Zigzag Conversion (LeetCode 6)
- Medium: Simplify Path (LeetCode 71)
- Hard: Text Justification (LeetCode 68) with precise space-distribution rules

PATTERN SUMMARY

Simulation problems reward careful, explicit case-by-case translation of real-world rules into code — there's no shortcut, only disciplined correctness.

PATTERN 40

Advanced Interview Problems (Synthesis)

FAANG Frequency	Extremely High
LeetCode Importance	Very High
Codeforces Importance	High
Companies Asking	Google, Amazon, Meta, Microsoft — senior/staff level rounds
Difficulty	Hard
Time Complexity	Varies — typically $O(n)$ to $O(n \log n)$
Space Complexity	Varies
Problems Covered	Word Ladder (string + BFS), Alien Dictionary (string + topological sort), Palindrome Pairs (string + Trie/hashing)

INTUITION

- The hardest FAANG string interviews rarely test ONE isolated pattern — they test whether you can COMBINE multiple patterns from this book: strings as graph nodes (Word Ladder), strings defining a graph's edges (Alien Dictionary), or strings requiring both hashing AND careful reversal logic (Palindrome Pairs).
- Recognizing which 2-3 patterns from Patterns 1-39 compose together is precisely the 'senior signal' interviewers are screening for at this level — identify the sub-problems explicitly before coding.

PATTERN RECOGNITION — Trigger Words

- strings that implicitly define a graph (transformation rules, ordering constraints)
- combine two or more earlier patterns
- senior/staff-level ambiguous framing requiring you to choose the decomposition

If interviewer says “given a list of words sorted according to an unknown alien alphabet, determine a valid ordering of the alphabet” — recognize this as Trie/prefix comparison (Pattern 18) building edges for a topological sort (graph algorithm, not a pure string pattern).

DRY RUN

```
Alien Dictionary: words=["wrt","wrf","er","ett","rftt"]
Compare adjacent words character-by-character to find the FIRST differing character -> establishes an edge (e.g. t->f from wrt/wrf)
Build a graph of these edges, topologically sort to get final alphabet order
```

BRUTE FORCE

- For graph-reduction problems, trying to reason about all constraints simultaneously without formally building a graph structure — leads to incorrect or incomplete orderings.

Time: Varies, often incorrect without formalization | Space: N/A

BETTER SOLUTION

- Correctly build the graph/edges but use a less efficient traversal (e.g., repeatedly scanning for zero-indegree nodes) instead of a proper queue-based topological sort.

Time: $O(V^2)$ instead of $O(V+E)$ | Space: $O(V+E)$

OPTIMAL SOLUTION

- Formally decompose the problem: (1) extract constraints from adjacent word comparisons in $O(\text{total characters})$, (2) build a directed graph, (3) run Kahn's algorithm (BFS-based topological sort) in $O(V+E)$.

Time: $O(C + V + E)$ where C = total characters | Space: $O(V+E)$

C++ CODE (Optimal)

pattern40_optimal.cpp

```
// Alien Dictionary skeleton: string comparison builds edges, then topological sort
string alienOrder(vector<string>& words) {
    unordered_map<char, unordered_set<char>> graph;
    unordered_map<char, int> indegree;
    for (auto &w : words) for (char c : w) indegree[c] = 0;

    for (int i = 0; i + 1 < (int)words.size(); i++) {
        string &a = words[i], &b = words[i + 1];
        int minLen = min(a.size(), b.size());
        bool found = false;
        for (int j = 0; j < minLen; j++) {
            if (a[j] != b[j]) {
                if (!graph[a[j]].count(b[j])) {
                    graph[a[j]].insert(b[j]);
                    indegree[b[j]]++;
                }
                found = true;
                break;
            }
        }
        // Edge case: "abc" before "ab" is invalid ordering (prefix must come first)
        if (!found && a.size() > b.size()) return "";
    }

    queue<char> q;
    for (auto &[c, d] : indegree) if (d == 0) q.push(c);
    string result;
    while (!q.empty()) {
        char c = q.front(); q.pop();
        result += c;
        for (char nxt : graph[c]) if (--indegree[nxt] == 0) q.push(nxt);
    }
    return result.size() == indegree.size() ? result : "";
}
```

COMMON MISTAKES

- Forgetting the classic edge case: if a longer word appears BEFORE its own prefix (e.g., "abc" before "ab"), the ordering is invalid — must explicitly detect and return failure.
- Only comparing the FIRST differing character between adjacent words — forgetting that characters AFTER the first difference carry no ordering information and must be ignored.
- Not detecting cycles in the resulting graph (a cycle means no valid ordering exists) — the topological sort's final result-length check against total node count is what catches this.
- Adding duplicate edges to the graph without checking existence first, causing indegree counts to be inflated incorrectly.
- For Word Ladder style problems, forgetting that this becomes a pure BFS shortest-path problem once you recognize each word as a graph node connected to words one edit-distance away.
- Not decomposing the problem explicitly before coding — senior interviews penalize jumping straight to code without first stating 'this reduces to problem X using technique Y'.
- Assuming a single pattern from earlier chapters solves the ENTIRE problem, missing that composition of 2+ patterns is required.
- For Palindrome Pairs, missing that BOTH directions must be checked (word A + word B AND word B + word A can both potentially form palindromes) and handling the empty-string edge case.

INTERVIEW TIPS

- Explicitly state your decomposition BEFORE coding: 'I'll treat this as a graph problem where strings define edges, then apply topological sort' — this sentence alone often is the single biggest signal in senior interviews.
- For any problem combining strings with graphs, mention both the string-processing complexity (usually $O(\text{total characters})$) and the graph-algorithm complexity ($O(V+E)$) separately.
- Always check for cycles / contradictions explicitly (topological sort's result length vs total distinct character count) — real inputs can be invalid, and correctly detecting that is part of the problem.
- When stuck, explicitly enumerate: 'is this a graph problem in disguise? A DP problem in disguise? A pure simulation?' — this is the master meta-skill this entire notes set has been building toward.

FOLLOW-UP QUESTIONS

- How would you extend Alien Dictionary to handle ties (multiple valid orderings) and return all of them?
- How does Word Ladder's BFS complexity scale, and how would you optimize with bidirectional BFS?
- For Palindrome Pairs, how would a Trie-based approach compare to a hashmap-based approach in practice?

VARIATIONS

- Easy: Verifying an Alien Dictionary (LeetCode 953, validation only, no ordering construction)
- Medium: Word Ladder (LeetCode 127) via BFS over one-edit-distance word graph
- Hard: Alien Dictionary (LeetCode 269) full topological sort + Palindrome Pairs (LeetCode 336) via Trie/hashing

PATTERN SUMMARY

Senior-level string interviews test PATTERN COMPOSITION — explicitly decompose into the 2-3 sub-patterns (graph, DP, hashing, Trie) at play before writing a single line of code.